

Table of Contents

Table of Contents	1
Contributors	12
1 CO & Architecture (251)	13
Topic-wise Key Concepts	13
Addressing Modes	13
Important Formulas & Concepts	13
Key Properties & Identities	13
Common Pitfalls	13
Standard Problem-Solving Techniques	13
Average Memory Access Time (AMAT)	13
Important Formulas & Concepts	14
Key Properties & Identities	14
Common Pitfalls	14
Standard Problem-Solving Techniques	14
Bit Vector	14
Important Formulas & Concepts	14
Key Properties & Identities	14
Common Pitfalls	14
Standard Problem-Solving Techniques	14
CISC RISC Architecture	14
Important Formulas & Concepts	14
Key Properties & Identities	15
Common Pitfalls	15
Standard Problem-Solving Techniques	15
Cache Memory	15
Important Formulas & Concepts	15
Key Properties & Identities	15
Common Pitfalls	16
Standard Problem-Solving Techniques	16
Conflict Misses	16
Important Formulas & Concepts	16
Key Properties & Identities	16
Common Pitfalls	16
Standard Problem-Solving Techniques	16
Control Unit	16
Important Formulas & Concepts	16
Key Properties & Identities	16
Common Pitfalls	16
Standard Problem-Solving Techniques	17
DMA (Direct Memory Access)	17
Important Formulas & Concepts	17
Key Properties & Identities	17
Common Pitfalls	17
Standard Problem-Solving Techniques	17
DRAM (Dynamic Random Access Memory)	17
Important Formulas & Concepts	17
Key Properties & Identities	17
Common Pitfalls	17
Standard Problem-Solving Techniques	17
Data Dependency	17
Important Formulas & Concepts	17
Key Properties & Identities	18
Common Pitfalls	18
Standard Problem-Solving Techniques	18
Data Hazards	18
Important Formulas & Concepts	18
Key Properties & Identities	18
Common Pitfalls	18
Standard Problem-Solving Techniques	18
Data Path	18
Important Formulas & Concepts	18
Key Properties & Identities	18
Common Pitfalls	19
Standard Problem-Solving Techniques	19
Direct Mapping	19

Important Formulas & Concepts	19
Key Properties & Identities	19
Common Pitfalls	19
Standard Problem-Solving Techniques	19
Dirty Bit	19
Important Formulas & Concepts	19
Key Properties & Identities	19
Common Pitfalls	19
Standard Problem-Solving Techniques	20
Disk	20
Important Formulas & Concepts	20
Key Properties & Identities	20
Common Pitfalls	20
Standard Problem-Solving Techniques	20
Hazards	20
Important Formulas & Concepts	20
Key Properties & Identities	20
Common Pitfalls	21
Standard Problem-Solving Techniques	21
O Handling	21
Important Formulas & Concepts	21
Key Properties & Identities	21
Common Pitfalls	21
Standard Problem-Solving Techniques	21
Instruction Execution	21
Important Formulas & Concepts	21
Key Properties & Identities	21
Common Pitfalls	21
Standard Problem-Solving Techniques	22
Instruction Format	22
Important Formulas & Concepts	22
Key Properties & Identities	22
Common Pitfalls	22
Standard Problem-Solving Techniques	22
Instruction Set Architecture (ISA)	22
Important Formulas & Concepts	22
Key Properties & Identities	22
Common Pitfalls	22
Standard Problem-Solving Techniques	22
Interrupts	22
Important Formulas & Concepts	23
Key Properties & Identities	23
Common Pitfalls	23
Standard Problem-Solving Techniques	23
Machine Instruction	23
Important Formulas & Concepts	23
Key Properties & Identities	23
Common Pitfalls	23
Standard Problem-Solving Techniques	23
Memory Interfacing	23
Important Formulas & Concepts	23
Key Properties & Identities	24
Common Pitfalls	24
Standard Problem-Solving Techniques	24
Microprogramming	24
Important Formulas & Concepts	24
Key Properties & Identities	24
Common Pitfalls	24
Standard Problem-Solving Techniques	24
Pipelining	24
Important Formulas & Concepts	24
Key Properties & Identities	25
Common Pitfalls	25
Standard Problem-Solving Techniques	25
Runtime Environment	25
Important Formulas & Concepts	25
Key Properties & Identities	25
Common Pitfalls	25
Standard Problem-Solving Techniques	25

Speedup	25
Important Formulas & Concepts	25
Key Properties & Identities	26
Common Pitfalls	26
Standard Problem-Solving Techniques	26
Stall	26
Important Formulas & Concepts	26
Key Properties & Identities	26
Common Pitfalls	26
Standard Problem-Solving Techniques	26
Virtual Memory	26
Important Formulas & Concepts	26
Key Properties & Identities	27
Common Pitfalls	27
Standard Problem-Solving Techniques	27
Quick Formula Reference	27
Important Tips for GATE	27
1.1 Addressing Modes (19)	28
1.2 Average Memory Access Time (3)	32
1.3 Bit Vector (1)	33
1.4 CISC RISC Architecture (2)	34
1.5 Cache Memory (69)	34
1.6 Conflict Misses (1)	50
1.7 Control Unit (1)	50
1.8 DMA (8)	50
1.9 DRAM (1)	52
1.10 Data Dependency (4)	52
1.11 Data Hazards (1)	53
1.12 Data Path (7)	54
1.13 Direct Mapping (5)	57
1.14 Disk (1)	58
1.15 Hazards (1)	58
1.16 IO Handling (8)	59
1.17 Instruction Execution (7)	61
1.18 Instruction Format (11)	62
1.19 Instruction Set Architecture (1)	65
1.20 Interrupts (10)	65
1.21 Machine Instruction (21)	67
1.22 Memory Interfacing (6)	75
1.23 Microprogramming (12)	76
1.24 Pipelining (39)	80
1.25 Runtime Environment (2)	90
1.26 Speedup (6)	91
1.27 Stall (1)	92
1.28 Virtual Memory (3)	92
Answer Keys	93
2 Computer Networks (226)	95
Topic-wise Key Concepts	95
Application Layer Protocols	95
Key Concepts:	95
Common Pitfalls and Problem-Solving Techniques:	95
ARP (Address Resolution Protocol)	95
Key Concepts:	95
Common Pitfalls and Problem-Solving Techniques:	96
Bit Stuffing	96
Key Concepts:	96
Common Pitfalls and Problem-Solving Techniques:	96
Bridges	96
Key Concepts:	96
Common Pitfalls and Problem-Solving Techniques:	96
CRC Polynomial (Cyclic Redundancy Check)	96
Important Formulas and Results:	96
Key Properties and Identities:	96

Common Pitfalls and Problem-Solving Techniques:	97
CSMA/CD (Carrier Sense Multiple Access with Collision Detection)	97
Important Formulas and Results:	97
Key Properties and Identities:	97
Common Pitfalls and Problem-Solving Techniques:	97
Channel Utilization	97
Important Formulas and Results:	97
Common Pitfalls and Problem-Solving Techniques:	98
Communication	98
Key Concepts:	98
Common Pitfalls and Problem-Solving Techniques:	98
Congestion Control	98
Key Concepts (TCP):	98
Common Pitfalls and Problem-Solving Techniques:	98
Data Communication	98
Key Concepts:	99
Common Pitfalls and Problem-Solving Techniques:	99
Distance Vector Routing	99
Key Concepts:	99
Common Pitfalls and Problem-Solving Techniques:	99
Error Detection	99
Key Concepts:	99
Important Formulas and Results:	99
Common Pitfalls and Problem-Solving Techniques:	99
Ethernet	99
Key Concepts:	100
Important Formulas and Results:	100
Common Pitfalls and Problem-Solving Techniques:	100
Fragmentation	100
Important Formulas and Results:	100
Key Properties and Identities:	100
Common Pitfalls and Problem-Solving Techniques:	100
Hamming Code	100
Important Formulas and Results:	100
Common Pitfalls and Problem-Solving Techniques:	101
IP Addressing	101
Important Formulas and Results (IPv4):	101
Key Properties and Identities:	101
Common Pitfalls and Problem-Solving Techniques:	101
IP Packet	101
Key Concepts (IPv4 Header Fields):	101
Common Pitfalls and Problem-Solving Techniques:	101
ICMP (Internet Control Message Protocol)	102
Key Concepts:	102
Common Pitfalls and Problem-Solving Techniques:	102
LAN Technologies	102
Key Concepts:	102
Common Pitfalls and Problem-Solving Techniques:	102
MAC Protocol (Medium Access Control)	102
Key Concepts:	102
Important Formulas and Results:	102
Common Pitfalls and Problem-Solving Techniques:	102
Network Flow	103
Key Concepts:	103
Common Pitfalls and Problem-Solving Techniques:	103
Network Layer	103
Key Concepts:	103
Common Pitfalls and Problem-Solving Techniques:	103
Network Protocols	103
Key Concepts:	103
Common Pitfalls and Problem-Solving Techniques:	103
Network Switching	103
Key Concepts:	104
Common Pitfalls and Problem-Solving Techniques:	104
OSI Model	104
Key Concepts:	104
Common Pitfalls and Problem-Solving Techniques:	104

Probability	104
Key Concepts:	104
Important Formulas and Results:	104
Common Pitfalls and Problem-Solving Techniques:	104
Pure Aloha	105
Important Formulas and Results:	105
Key Properties and Identities:	105
Common Pitfalls and Problem-Solving Techniques:	105
Routing	105
Key Concepts:	105
Common Pitfalls and Problem-Solving Techniques:	105
Routing Protocols	105
Key Concepts:	105
Common Pitfalls and Problem-Solving Techniques:	106
Sliding Window	106
Important Formulas and Results:	106
Key Properties and Identities:	106
Common Pitfalls and Problem-Solving Techniques:	106
Slotted Aloha	106
Important Formulas and Results:	106
Key Properties and Identities:	107
Common Pitfalls and Problem-Solving Techniques:	107
Sockets	107
Key Concepts:	107
Common Pitfalls and Problem-Solving Techniques:	107
Stop and Wait	107
Important Formulas and Results:	107
Key Properties and Identities:	107
Common Pitfalls and Problem-Solving Techniques:	108
Subnetting	108
Important Formulas and Results:	108
Common Pitfalls and Problem-Solving Techniques:	108
TCP (Transmission Control Protocol)	108
Key Concepts:	108
Important Formulas and Results:	108
Common Pitfalls and Problem-Solving Techniques:	108
Token Bucket	108
Key Concepts:	109
Important Formulas and Results:	109
Common Pitfalls and Problem-Solving Techniques:	109
UDP (User Datagram Protocol)	109
Key Concepts:	109
Key Properties and Identities:	109
Common Pitfalls and Problem-Solving Techniques:	109
Wrap Around Time	109
Important Formulas and Results:	109
Key Properties and Identities:	110
Common Pitfalls and Problem-Solving Techniques:	110
Quick Formula Reference	110
Channel Utilization & Throughput:	110
Error Detection & Correction:	110
MAC Protocols:	110
IP Fragmentation:	110
IP Addressing & Subnetting:	110
Sliding Window Sequence Numbers:	110
Wrap Around Time:	110
Important Tips for GATE	110
2.1 Application Layer Protocols (13)	111
2.2 Arp (1)	113
2.3 Bit Stuffing (2)	114
2.4 Bridges (3)	114
2.5 CRC Polynomial (5)	116
2.6 CSMA CD (6)	117
2.7 Channel Utilization (1)	118
2.8 Communication (4)	118
2.9 Congestion Control (9)	119
2.10 Data Communication (1)	121

2.11 Distance Vector Routing (8)	121
2.12 Error Detection (8)	125
2.13 Ethernet (7)	127
2.14 Fragmentation (5)	129
2.15 Hamming Code (2)	130
2.16 IP Addressing (8)	131
2.17 IP Packet (12)	132
2.18 Icmp (1)	135
2.19 LAN Technologies (7)	135
2.20 MAC Protocol (4)	136
2.21 Network Flow (4)	137
2.22 Network Layer (6)	138
2.23 Network Protocols (11)	139
2.24 Network Switching (4)	142
2.25 Osi Model (1)	143
2.26 Probability (1)	143
2.27 Pure Aloha (1)	143
2.28 Routing (14)	143
2.29 Routing Protocols (1)	148
2.30 Sliding Window (16)	148
2.31 Slotted Aloha (1)	151
2.32 Sockets (4)	151
2.33 Stop and Wait (6)	152
2.34 Subnetting (21)	153
2.35 TCP (20)	159
2.36 Token Bucket (2)	163
2.37 UDP (4)	163
2.38 Wrap Around Time (2)	164
Answer Keys	165
3 Databases (302)	167
Topic-wise Key Concepts	167
Armstrong Axioms	167
Formulas/Theorems:	167
Key Properties/Identities:	167
Common Pitfalls:	167
Problem-Solving Techniques:	167
B Tree	167
Formulas/Theorems:	168
Key Properties/Identities:	168
Common Pitfalls:	168
Problem-Solving Techniques:	168
Candidate Key	168
Formulas/Theorems:	168
Key Properties/Identities:	168
Common Pitfalls:	168
Problem-Solving Techniques:	168
Conflict Serializable	168
Formulas/Theorems:	169
Key Properties/Identities:	169
Common Pitfalls:	169
Problem-Solving Techniques:	169
Database Design	169
Formulas/Theorems:	169
Key Properties/Identities:	169
Common Pitfalls:	169
Problem-Solving Techniques:	169
Database Normalization	169
Formulas/Theorems:	169
Key Properties/Identities:	170
Common Pitfalls:	170
Problem-Solving Techniques:	170
Database Schema	170
Formulas/Theorems:	170
Key Properties/Identities:	170

Common Pitfalls:	170
Problem-Solving Techniques:	170
Decomposition	170
Formulas/Theorems:	170
Key Properties/Identities:	171
Common Pitfalls:	171
Problem-Solving Techniques:	171
ER Diagram	171
Formulas/Theorems:	171
Key Properties/Identities:	171
Common Pitfalls:	171
Problem-Solving Techniques:	171
Functional Dependency	171
Formulas/Theorems:	171
Key Properties/Identities:	172
Common Pitfalls:	172
Problem-Solving Techniques:	172
Indexing	172
Formulas/Theorems:	172
Key Properties/Identities:	172
Common Pitfalls:	172
Problem-Solving Techniques:	172
Joins	172
Formulas/Theorems:	172
Key Properties/Identities:	173
Common Pitfalls:	173
Problem-Solving Techniques:	173
Multivalued Dependency 4NF	173
Formulas/Theorems:	173
Key Properties/Identities:	173
Common Pitfalls:	173
Problem-Solving Techniques:	173
Natural Join	174
Formulas/Theorems:	174
Key Properties/Identities:	174
Common Pitfalls:	174
Problem-Solving Techniques:	174
Normal Forms	174
Formulas/Theorems:	174
Key Properties/Identities:	174
Common Pitfalls:	174
Problem-Solving Techniques:	174
Query	175
Formulas/Theorems:	175
Key Properties/Identities:	175
Common Pitfalls:	175
Problem-Solving Techniques:	175
Referential Integrity	175
Formulas/Theorems:	175
Key Properties/Identities:	175
Common Pitfalls:	175
Problem-Solving Techniques:	175
Relational Algebra	175
Formulas/Theorems:	176
Key Properties/Identities:	176
Common Pitfalls:	176
Problem-Solving Techniques:	176
Relational Calculus	176
Formulas/Theorems:	176
Key Properties/Identities:	176
Common Pitfalls:	176
Problem-Solving Techniques:	176
Relational Model	177
Formulas/Theorems:	177
Key Properties/Identities:	177
Common Pitfalls:	177
Problem-Solving Techniques:	177
SQL	177
Formulas/Theorems:	177

Key Properties/Identities:	177
Common Pitfalls:	177
Problem-Solving Techniques:	177
Safe Query	177
Formulas/Theorems:	178
Key Properties/Identities:	178
Common Pitfalls:	178
Problem-Solving Techniques:	178
Super Key	178
Formulas/Theorems:	178
Key Properties/Identities:	178
Common Pitfalls:	178
Problem-Solving Techniques:	178
Timestamp Ordering	178
Formulas/Theorems:	178
Key Properties/Identities:	179
Common Pitfalls:	179
Problem-Solving Techniques:	179
Transaction and Concurrency	179
Formulas/Theorems:	179
Key Properties/Identities:	179
Common Pitfalls:	179
Problem-Solving Techniques:	179
Tuple Relational Calculus	179
Formulas/Theorems:	179
Key Properties/Identities:	180
Common Pitfalls:	180
Problem-Solving Techniques:	180
Two Phase Locking Protocol	180
Formulas/Theorems:	180
Key Properties/Identities:	180
Common Pitfalls:	180
Problem-Solving Techniques:	180
Quick Formula Reference	180
Important Tips for GATE	181
3.1 Armstrong Axioms (1)	181
3.2 B Tree (32)	182
3.3 Candidate Key (7)	190
3.4 Conflict Serializable (12)	191
3.5 Database Design (1)	195
3.6 Database Normalization (56)	195
3.7 Database Schema (1)	208
3.8 Decomposition (1)	208
3.9 ER Diagram (12)	209
3.10 Functional Dependency (3)	212
3.11 Indexing (15)	213
3.12 Joins (7)	216
3.13 Multivalued Dependency 4nf (1)	218
3.14 Natural Join (3)	218
3.15 Normal Forms (1)	219
3.16 Query (3)	219
3.17 Referential Integrity (6)	221
3.18 Relational Algebra (33)	223
3.19 Relational Calculus (13)	234
3.20 Relational Model (2)	237
3.21 SQL (58)	237
3.22 Safe Query (1)	260
3.23 Super Key (1)	261
3.24 Timestamp Ordering (1)	261
3.25 Transaction and Concurrency (27)	261
3.26 Tuple Relational Calculus (3)	271
3.27 Two Phase Locking Protocol (1)	272
Answer Keys	272
4 Digital Logic (313)	275

Topic-wise Key Concepts	275
Adder	275
Array Multiplier	276
Binary Codes	276
Boolean Algebra	276
Booths Algorithm	277
Canonical Normal Form	277
Carry Generator	277
Circuit Output	278
Combinational Circuit	278
Conjunctive Normal Form	278
Decoder	278
Digital Circuits	279
Digital Counter	279
Dual Function	279
Finite State Machines (FSM)	279
Fixed Point Representation	280
Flip Flop	280
Floating Point Representation	281
Functional Completeness	281
IEEE Representation	281
K Map (Karnaugh Map)	282
Little Endian Big Endian	282
Memory Interfacing	282
Min No Gates	283
Min Products of Sum Form (MPOS)	283
Min Sum of Products Form (MSOP)	283
Multiplexer (MUX)	283
Number Representation	284
Number System	284
Prime Implicants	284
ROM (Read-Only Memory)	285
Reduction	285
Ripple Counter Operation	285
Shift Registers	285
Static Hazard	286
Synchronous Asynchronous Circuits	286
Quick Formula Reference	286
Important Tips for GATE	287
4.1 Adder (9)	287
4.2 Array Multiplier (2)	289
4.3 Binary Codes (1)	290
4.4 Boolean Algebra (34)	290
4.5 Booths Algorithm (7)	296
4.6 Canonical Normal Form (10)	297
4.7 Carry Generator (2)	300
4.8 Circuit Output (40)	301
4.9 Combinational Circuit (2)	314
4.10 Conjunctive Normal Form (1)	315
4.11 Decoder (3)	316
4.12 Digital Circuits (7)	316
4.13 Digital Counter (18)	318
4.14 Dual Function (1)	323
4.15 Finite State Machines (4)	323
4.16 Fixed Point Representation (2)	324
4.17 Flip Flop (7)	325
4.18 Floating Point Representation (8)	327
4.19 Functional Completeness (7)	329
4.20 IEEE Representation (14)	330
4.21 K Map (17)	333

4.22 Little Endian Big Endian (1)	338
4.23 Memory Interfacing (5)	339
4.24 Min No Gates (6)	339
4.25 Min Products of Sum Form (2)	341
4.26 Min Sum of Products Form (16)	341
4.27 Multiplexer (14)	345
4.28 Number Representation (57)	348
4.29 Number System (1)	358
4.30 Prime Implicants (2)	358
4.31 ROM (4)	358
4.32 Reduction (1)	359
4.33 Ripple Counter Operation (1)	359
4.34 Shift Registers (2)	359
4.35 Static Hazard (1)	360
4.36 Synchronous Asynchronous Circuits (4)	360
Answer Keys	361
5 Operating System (343)	364
Topic-wise Key Concepts	364
Bankers Algorithm	364
Best Fit	364
Context Switch	365
DMA (Direct Memory Access)	365
Deadlock Prevention Avoidance Detection	365
Demand Paging	366
Disk	366
Disk Scheduling	367
File System	367
Fork System Call	367
IO Handling	368
Input Output	368
Inter Process Communication (IPC)	368
Interrupts	369
Least Recently Used (LRU)	369
Linked Allocation	369
Memory Management	369
Multilevel Paging	370
OS Protection	370
Page Replacement	370
Precedence Graph	371
Process	371
Process Scheduling	372
Process Synchronization	372
Resource Allocation	373
Resource Allocation Graph (RAG)	373
Round Robin Scheduling	373
Semaphore	374
SRTF (Shortest Remaining Time First)	374
System Calls	375
Threads	375
Translation Lookaside Buffer (TLB)	375
Virtual Memory	376
Quick Formula Reference	376
Important Tips for GATE	376
5.1 Bankers Algorithm (2)	377
5.2 Best Fit (1)	378
5.3 Context Switch (4)	378
5.4 DMA (1)	379
5.5 Deadlock Prevention Avoidance Detection (4)	379
5.6 Demand Paging (3)	380
5.7 Disk (30)	381

5.8 Disk Scheduling (16)	387
5.9 File System (7)	390
5.10 Fork System Call (8)	392
5.11 IO Handling (7)	394
5.12 Input Output (1)	395
5.13 Inter Process Communication (1)	396
5.14 Interrupts (6)	396
5.15 Least Recently Used (1)	397
5.16 Linked Allocation (1)	398
5.17 Memory Management (9)	398
5.18 Multilevel Paging (1)	400
5.19 OS Protection (3)	400
5.20 Page Replacement (31)	401
5.21 Precedence Graph (3)	407
5.22 Process (5)	408
5.23 Process Scheduling (49)	409
5.24 Process Synchronization (52)	421
5.25 Resource Allocation (27)	442
5.26 Resource Allocation Graph (1)	451
5.27 Round Robin Scheduling (1)	451
5.28 Semaphore (11)	451
5.29 Srtf (1)	455
5.30 System Calls (1)	455
5.31 Threads (10)	455
5.32 Translation Lookaside Buffer (2)	458
5.33 Virtual Memory (43)	458
Answer Keys	467

Contributors

User	Answers	User	?Added	User	Done
Arjun Suresh	21202, 265	Kathleen Bankson	470	Lakshman Patel (GATE	639
Akash Kanase	3338, 44	Arjun Suresh	302	And Tech)	
Sachin Mittal	2663, 22	GO Editor	226	Arjun Suresh	331
Rajarshi Sarkar	2642, 46	Ishrat Jahan	194	kenzou	248
Digvijay	2546, 32	gatecse	105	Milicevic3306	178
Amar Vashishth	2520, 30	Misbah	77	Pooja Khatri	151
Akhil Nadh PC	1796, 11	Rucha Shelke	34	Ajay kumar soni	110
Vikrant Singh	1653, 18	admin	32	Naveen Kumar	99
Deepak Poonia	1557, 21	Sandeep Singh	27	Hira	72
Praveen Saini	1432, 26	Akash Kanase	22	gatecse	71
Pooja Palod	1385, 20	Madhav kumar	10	Shubham Sharma	69
Sandeep Uniyal	1315, 14	khush tak	7	GO Editor	59
Shaik Masthan	1223, 28			Shri 1	56
gatecse	1215, 18			Misbah	54
Sankaranarayanan P.N	1181, 17			Shaik Masthan	41
Ayush Upadhyaya	1132, 15			shadymademe	35
Manu Thakur	1128, 19			Kavali Yedidyah Sagar	32
Gate Keeda	1084, 20			dj_1	30
Prashant Singh	1003, 18			Deepak Poonia	28
Aravind	940, 14			Pavan Singh	26
dd	909, 4			Akash Dinkar	26
Himanshu Agarwal	890, 8			GO Classes for GATE	23
minal	871, 13			CSE	
Sourav Roy	843, 13			Sweta Kumari Rawani	21
neha pawar	842, 12			Krithiga2101	20
Abhilash Panicker	765, 9			iothee_new	17
Kalpish Singhal	760, 6			Sukanya Das	16
jayendra	711, 12			Sachin Mittal	14
Mithlesh Upadhyay	616, 5			soujanyareddy13	13
Vicky Bajoria	616, 9			Subarna Das	13
Danish	548, 6			Shikha Mallick	12
Hira	525, 21			Manoja Rajalakshmi	11
Anurag Semwal	508, 7			Aravindakshan	
Debasish Das	502, 5			srestha	10
Ahwan Mishra	492, 5			Abhrajyoti Kundu	10
Anoop Sonkar	472, 7			CS For	10
srestha	459, 8			HIVA	
				Prof Walter White	10
				Satbir Singh	9
				KUSHAGRA गुप्ता	9
				Umesh Shelke	9
				Pooja Palod	8
				Gyanendra Singh	8
				Shiva Sagar Rao	8
				Rajarshi Sarkar	7



Machine instructions and Addressing modes, ALU, data-path and control unit, Instruction pipelining, Pipeline hazards, Memory hierarchy: cache, main memory and secondary storage; I/O interface (Interrupt and DMA mode)

Mark Distribution in Previous GATE

Year	2026 - 1	2026 - 2	2025 - 1	2025 - 2	2024 - 1	2024 - 2	2023	2022	2021 - 1	2021 - 2	Minimum
1 Mark Count	3	1	2	1	2	2	2	3	1	2	1
2 Marks Count	4	5	3	4	3	3	4	2	2	2	2
Total Marks	11	11	8	9	8	8	10	7	5	6	5

Welcome to the "Computer Organization & Architecture" chapter of your GATE Computer Science preparation. This crucial subject delves into the fundamental design and operational principles of a computer system, from the lowest-level hardware components to how instructions are executed and data is managed. Understanding COA is vital for any computer scientist as it provides the bedrock knowledge for designing efficient systems, writing optimized code, and comprehending the performance implications of software choices. In the GATE CS exam, COA typically carries a weightage of 8-12 marks, with questions ranging from conceptual understanding of architectural principles (like CISC vs. RISC, control unit types) to numerical problems involving cache memory, pipelining performance, virtual memory, and disk access times. Expect a mix of Multiple Choice Questions (MCQs), Multiple Select Questions (MSQs), and Numerical Answer Type (NAT) questions, often requiring precise calculations and a deep grasp of underlying mechanisms.

Topic-wise Key Concepts

Addressing Modes

Addressing modes define how the effective address of an operand is calculated. They specify where the operand is located, whether in a register, memory, or as part of the instruction itself, enabling flexible and efficient memory access.

Important Formulas & Concepts

- Immediate:** Operand is part of the instruction. $EA = \text{Operand Value}$. No memory access for operand.
- Direct:** $EA = \text{Address}$ (address field in instruction). One memory access to fetch operand.
- Indirect:** $EA = M[\text{Address}]$ (address field points to memory location containing EA). Two memory accesses (one for EA, one for operand).
- Register:** Operand is in a specified register. $EA = R$. No memory access.
- Register Indirect:** $EA = [R]$ (register contains EA). One memory access to fetch operand.
- Relative (PC-relative):** $EA = PC + \text{Offset}$ (offset from Program Counter). Used for branch instructions.
- Indexed:** $EA = \text{Base_Address} + \text{Index_Register}$. Used for array access.
- Base-Register:** $EA = \text{Base_Register} + \text{Offset}$. Used for relocating programs.

Key Properties & Identities

- Trade-offs exist between instruction length, execution speed, and flexibility.
- Used to access data, implement control flow, and manage memory.

Common Pitfalls

- Confusing direct and indirect addressing, especially regarding the number of memory accesses.

Standard Problem-Solving Techniques

- Trace the instruction execution step-by-step to calculate the effective address for given register/memory contents.

Average Memory Access Time (AMAT)

AMAT is the average time taken to access memory, considering the presence of a cache and its hit/miss rates. It quantifies the performance of the memory hierarchy.

Important Formulas & Concepts

- For a single-level cache:

$$AMAT = T_{hit} + MR \times T_{miss_penalty}$$

where T_{hit} is cache hit time, MR is miss rate, and $T_{miss_penalty}$ is the time to fetch data from the next level of memory (main memory).

- For a two-level cache (L1, L2):

$$AMAT = T_{L1_hit} + MR_{L1} \times (T_{L2_hit} + MR_{L2} \times T_{main_memory})$$

where T_{L1_hit} is L1 hit time, MR_{L1} is L1 miss rate, T_{L2_hit} is L2 hit time, MR_{L2} is L2 miss rate (local miss rate for L2), and T_{main_memory} is main memory access time.

Key Properties & Identities

- Lower AMAT indicates better memory system performance.
- Affected by cache hit rate, miss penalty, and cache access times.

Common Pitfalls

- Forgetting to add the hit time to the miss penalty when calculating total miss time.
- Incorrectly applying local vs. global miss rates for multi-level caches.

Standard Problem-Solving Techniques

- Draw the memory hierarchy and clearly identify access times and miss rates for each level.

Bit Vector

A bit vector (or bit array) is a data structure that efficiently stores a sequence of bits. Each bit typically represents a boolean flag or the presence/absence of an element in a set.

Important Formulas & Concepts

- Storage: N bits require $\lceil N/8 \rceil$ bytes of storage.
- Bitwise operations: AND, OR, XOR, NOT, SHIFT for efficient manipulation.

Key Properties & Identities

- Space-efficient for large sets of boolean flags.
- Fast bitwise operations for checking, setting, or clearing flags.

Common Pitfalls

- Misinterpreting bit positions or the effect of bitwise operations.

Standard Problem-Solving Techniques

- Use bit masks to isolate or modify specific bits.

CISC RISC Architecture

CISC (Complex Instruction Set Computer) and RISC (Reduced Instruction Set Computer) are two contrasting philosophies for designing instruction sets. They represent different approaches to instruction complexity and hardware implementation.

Important Formulas & Concepts

- CISC Characteristics:

- Many complex instructions, often performing multiple operations (e.g., memory access and arithmetic).
- Variable instruction length.
- Microprogrammed control unit.
- Fewer general-purpose registers, more memory-to-memory operations.
- Complex addressing modes.
- RISC Characteristics:**
 - Few, simple, fixed-length instructions, typically one operation per instruction.
 - Hardwired control unit.
 - Many general-purpose registers, load/store architecture (only LOAD/STORE access memory).
 - Simple addressing modes.
 - Optimized for pipelining.

Key Properties & Identities

- RISC generally achieves higher CPI (Cycles Per Instruction) but lower clock cycle time due to simpler instructions, leading to better performance in many cases.
- CISC aims for fewer instructions per program but higher CPI.

Common Pitfalls

- Assuming one architecture is universally superior; their suitability depends on the application and compiler technology.

Standard Problem-Solving Techniques

- Compare and contrast features to identify the type of architecture described in a scenario.

Cache Memory

Cache memory is a small, fast memory component that stores copies of data from frequently used main memory locations. Its purpose is to reduce the average time to access data, exploiting the principle of locality.

Important Formulas & Concepts

- Address Breakdown:** For a memory address of M bits, a cache with B bytes per block, and C cache lines (or sets):
 - Block Offset bits: $\log_2 B$
 - Number of Blocks in Main Memory: $2^M / B$
 - Number of Cache Lines: C
- Direct Mapping:**
 - Index bits: $\log_2 C$
 - Tag bits: $M - (\log_2 C + \log_2 B)$
 - Cache Line Index = $(\text{Main Memory Block Number}) \pmod{C}$
- Set-Associative Mapping (S-way):**
 - Number of Sets: C/S
 - Index bits: $\log_2(C/S)$
 - Tag bits: $M - (\log_2(C/S) + \log_2 B)$
 - Cache Set Index = $(\text{Main Memory Block Number}) \pmod{(C/S)}$
- Fully Associative Mapping:**
 - No Index bits
 - Tag bits: $M - \log_2 B$
 - Any block can go into any cache line.
- Hit Rate (HR):** $\frac{\text{Number of Cache Hits}}{\text{Total Memory Accesses}}$
- Miss Rate (MR):** $1 - HR$

Key Properties & Identities

- Exploits temporal and spatial locality of reference.
- Mapping techniques (Direct, Set-Associative, Fully Associative) determine where a block can be placed.
- Write policies (Write-Through, Write-Back) determine when data is written to main memory.

- Replacement policies (LRU, FIFO, LFU, Random) determine which block to evict on a miss.

Common Pitfalls

- Incorrectly calculating the number of bits for Tag, Index, and Offset.
- Confusing the total cache size with the number of cache lines.

Standard Problem-Solving Techniques

- Draw the cache structure and trace memory accesses to determine hits/misses and cache contents.

Conflict Misses

Conflict misses occur in direct-mapped or set-associative caches when multiple memory blocks map to the same cache line or set, and these blocks are actively used, causing them to evict each other even if other cache lines are empty.

Important Formulas & Concepts

- These are a type of "capacity miss" specifically due to the mapping function.
- Increasing associativity reduces conflict misses.

Key Properties & Identities

- Occur when the cache is not full but the required block is evicted due to mapping constraints.
- Distinguished from compulsory (first-time access) and capacity (cache too small) misses.

Common Pitfalls

- Mistaking a conflict miss for a capacity miss or a compulsory miss.

Standard Problem-Solving Techniques

- Analyze the memory access pattern and the cache mapping scheme to identify if multiple active blocks map to the same location.

Control Unit

The control unit is the part of the CPU that directs the operation of the processor. It interprets instructions and generates control signals to coordinate the activities of other components, such as the ALU, registers, and memory.

Important Formulas & Concepts

- **Hardwired Control:** Implemented using combinational logic gates.
 - Faster execution.
 - Complex to design and modify for large instruction sets.
 - Used in RISC processors.
- **Microprogrammed Control:** Implemented using a sequence of microinstructions stored in a control memory.
 - More flexible and easier to design/modify for complex instruction sets.
 - Slower execution due to memory access for microinstructions.
 - Used in CISC processors.

Key Properties & Identities

- Determines the sequence of micro-operations for each machine instruction.
- Generates timing and control signals for the entire system.

Common Pitfalls

- Not understanding the trade-offs between hardwired and microprogrammed control in terms of speed, flexibility, and design complexity.

Standard Problem-Solving Techniques

- Relate the control unit type to the characteristics of CISC/RISC architectures.

DMA (Direct Memory Access)

DMA is a hardware feature that allows I/O devices to transfer data directly to and from main memory without involving the CPU. This significantly reduces CPU overhead for data transfers, improving system performance.

Important Formulas & Concepts

- DMA controller manages the data transfer.
- **Cycle Stealing:** DMA controller takes control of the bus for one or more memory cycles to transfer data.
- **Burst Mode:** DMA controller takes control of the bus and transfers an entire block of data before releasing the bus.

Key Properties & Identities

- Frees the CPU to perform other tasks during I/O operations.
- Requires a DMA controller chip.

Common Pitfalls

- Misunderstanding that DMA completely bypasses the CPU; the CPU initiates and configures the DMA transfer.

Standard Problem-Solving Techniques

- Analyze scenarios to determine when DMA is most beneficial compared to programmed I/O or interrupt-driven I/O.

DRAM (Dynamic Random Access Memory)

DRAM is the most common type of main memory used in computers. It stores each bit of data in a separate capacitor within an integrated circuit, requiring periodic refreshing to maintain the charge and thus the data.

Important Formulas & Concepts

- Data stored as charge in capacitors.
- Requires refresh cycles to prevent data loss.

Key Properties & Identities

- Volatile (data is lost when power is removed).
- Higher density and lower cost per bit compared to SRAM.
- Slower access times than SRAM due to refresh cycles and capacitor charging/discharging.

Common Pitfalls

- Confusing DRAM characteristics with SRAM (Static RAM), which uses latches and does not require refreshing.

Standard Problem-Solving Techniques

- Compare and contrast DRAM with SRAM based on cost, speed, density, and power consumption.

Data Dependency

A data dependency exists between two instructions when the second instruction requires data produced by the first instruction, or when they both access the same memory location, potentially leading to incorrect results if executed out of order.

Important Formulas & Concepts

- **RAW (Read After Write) / True Dependency:** Instruction J tries to read a source before instruction I writes to it. $I \rightarrow J$. This is the most common and critical dependency.
- **WAR (Write After Read) / Anti-dependency:** Instruction J tries to write to a destination before instruction I reads from it. $I \leftarrow J$. Can be resolved by renaming.
- **WAW (Write After Write) / Output Dependency:** Instruction J tries to write to a destination before instruction I writes to it. $I \Rightarrow J$. Can be resolved by renaming.

Key Properties & Identities

- RAW dependencies are true data dependencies and must be preserved.
- WAR and WAW are name dependencies and can be eliminated by register renaming.

Common Pitfalls

- Incorrectly identifying the type of dependency or missing a dependency.

Standard Problem-Solving Techniques

- Analyze the read/write sets of instructions to identify dependencies.

Data Hazards

Data hazards occur in pipelined processors when an instruction attempts to access data before a preceding instruction has made that data available. These hazards can lead to incorrect program execution if not handled.

Important Formulas & Concepts

- Caused by RAW, WAR, and WAW dependencies.
- **Solutions:**
 - **Stalling (Bubbles/NOPs):** Insert NOP instructions to delay the dependent instruction.
 - **Forwarding (Bypassing):** Route the result of an instruction directly from an internal pipeline register to the input of a subsequent instruction's functional unit, bypassing the register file.
 - **Compiler Scheduling:** Reorder instructions at compile time to avoid hazards.

Key Properties & Identities

- Increase the CPI (Cycles Per Instruction) of a pipeline, reducing ideal speedup.
- Forwarding is the most common hardware solution to reduce stalls due to RAW hazards.

Common Pitfalls

- Calculating the number of stalls incorrectly, especially when forwarding is present.
- Not identifying all data dependencies in a given instruction sequence.

Standard Problem-Solving Techniques

- Draw pipeline diagrams to visualize instruction flow and identify where stalls or forwarding are needed.

Data Path

The data path of a CPU consists of the functional units (like the ALU, registers, and multiplexers) and the buses that connect them, through which data flows during instruction execution. It performs the actual data processing operations.

Important Formulas & Concepts

- Key components: Program Counter (PC), Instruction Register (IR), Memory Address Register (MAR), Memory Data Register (MDR), General Purpose Registers (GPRs), Arithmetic Logic Unit (ALU).

Key Properties & Identities

- The data path is controlled by signals generated by the control unit.
- Its design impacts the clock cycle time and the number of cycles per instruction.

Common Pitfalls

- Not understanding how data moves between components for different instruction types.

Standard Problem-Solving Techniques

- Trace the flow of data for a specific instruction (e.g., LOAD, ADD) through the datapath components.

Direct Mapping

Direct mapping is a cache organization technique where each block from main memory can only be placed into one specific line in the cache. This mapping is determined by a simple modular arithmetic function of the memory block address.

Important Formulas & Concepts

- **Cache Line Index:**

$$\text{Cache Line Index} = (\text{Main Memory Block Number}) \pmod{\text{Number of Cache Lines}}$$
- **Address Breakdown:** A memory address is divided into three fields: Tag, Index, and Block Offset.
 - **Block Offset bits:** $\log_2(\text{Block Size in bytes})$
 - **Index bits:** $\log_2(\text{Number of Cache Lines})$
 - **Tag bits:** $\text{Total Address Bits} - \text{Index Bits} - \text{Block Offset Bits}$

Key Properties & Identities

- Simplest to implement and fastest for cache lookup.
- Suffers from high conflict misses if frequently accessed blocks map to the same cache line.

Common Pitfalls

- Incorrectly calculating the number of bits for the Tag, Index, or Block Offset fields.

Standard Problem-Solving Techniques

- Given memory and cache parameters, determine the address breakdown and trace memory accesses to identify hits/misses.

Dirty Bit

A dirty bit (or modified bit) is a flag associated with a cache line or a virtual memory page table entry. It indicates whether the corresponding data block in the cache/page has been modified since it was loaded from the next lower level of memory (main memory/disk).

Important Formulas & Concepts

- Set to '1' when data in the cache line or page is written to.
- Set to '0' when the data is clean (matches the lower level memory).

Key Properties & Identities

- Crucial for write-back cache policies: if the dirty bit is set, the block must be written back to main memory upon eviction.
- Used in virtual memory page replacement algorithms to avoid unnecessary writes to disk.

Common Pitfalls

- Misunderstanding its role in write-through caches (where it's not typically needed) versus write-back caches.

Standard Problem-Solving Techniques

- Analyze cache write operations and page replacement scenarios to determine when the dirty bit is set or checked.

Disk

A disk (hard disk drive) is a non-volatile secondary storage device that stores data magnetically on rotating platters. It is characterized by its large capacity and persistence, but significantly slower access times compared to RAM.

Important Formulas & Concepts

- Disk Access Time:**

$$Access_Time = Seek_Time + Rotational_Latency + Transfer_Time$$

- Seek Time:** Time taken for the read/write head to move to the correct track. (Often given as average or calculated based on number of tracks).
- Rotational Latency:** Time taken for the desired sector to rotate under the read/write head.

$$Average_Rotational_Latency = \frac{1}{2} \times \frac{60 \text{ seconds}}{\text{RPM}}$$

(where RPM is revolutions per minute).

- Transfer Time:** Time taken to transfer the actual data once the head is positioned.

$$Transfer_Time = \frac{\text{Number of Sectors to Transfer}}{\text{Sectors per Track}} \times \frac{60 \text{ seconds}}{\text{RPM}}$$

$$\text{or } Transfer_Time = \frac{\text{Amount of Data}}{\text{Transfer Rate}}.$$

Key Properties & Identities

- Non-volatile storage.
- Mechanical components make it orders of magnitude slower than electronic memory.

Common Pitfalls

- Incorrectly calculating average rotational latency (using $1/2$ of a full rotation).
- Mixing units (e.g., milliseconds and seconds).

Standard Problem-Solving Techniques

- Break down the access time calculation into its three components and calculate each separately.

Hazards

Hazards are situations in pipelined processors that prevent the next instruction in the instruction stream from executing in its designated clock cycle. They reduce the ideal speedup of a pipeline.

Important Formulas & Concepts

- Structural Hazards:** Occur when two instructions require the same hardware resource at the same time (e.g., single memory port for both instruction fetch and data access).
- Data Hazards:** Occur when an instruction depends on the result of a previous instruction that has not yet completed (RAW, WAR, WAW).
- Control Hazards (Branch Hazards):** Occur when the pipeline makes a wrong guess about the next instruction to fetch (e.g., after a conditional branch).

Key Properties & Identities

- All hazards increase the effective CPI (Cycles Per Instruction) of the pipeline.

- Solutions include stalling, forwarding, and branch prediction.

Common Pitfalls

- Confusing the different types of hazards or misidentifying the cause of a stall.

Standard Problem-Solving Techniques

- Analyze the instruction sequence and pipeline stage usage to identify potential conflicts.

I/O Handling

I/O handling refers to the mechanisms by which a CPU communicates and exchanges data with peripheral input/output devices. Efficient I/O handling is crucial for system performance.

Important Formulas & Concepts

- **Programmed I/O (Polling):** CPU continuously checks the status of an I/O device.
 - High CPU overhead, simple to implement.
- **Interrupt-Driven I/O:** I/O device signals the CPU via an interrupt when it needs attention.
 - Lower CPU overhead than polling, CPU can do other work.
 - Requires interrupt service routines (ISRs).
- **Direct Memory Access (DMA):** I/O device transfers data directly to/from memory without CPU intervention.
 - Lowest CPU overhead for large data transfers.
 - Requires a DMA controller.

Key Properties & Identities

- Each method has trade-offs in terms of CPU utilization, response time, and complexity.
- Interrupts allow for asynchronous event handling.

Common Pitfalls

- Not understanding the CPU's involvement (or lack thereof) in data transfer for each method.

Standard Problem-Solving Techniques

- Evaluate scenarios to determine the most suitable I/O handling method based on data volume and CPU availability.

Instruction Execution

Instruction execution is the process by which a CPU carries out a single machine instruction. In a pipelined processor, this process is broken down into several sequential stages, allowing multiple instructions to be processed concurrently.

Important Formulas & Concepts

- **Typical 5-stage pipeline:**
 1. **IF (Instruction Fetch):** Fetch instruction from memory.
 2. **ID (Instruction Decode):** Decode instruction, read registers.
 3. **EX (Execute):** Perform ALU operation.
 4. **MEM (Memory Access):** Access data memory (load/store).
 5. **WB (Write Back):** Write result back to register file.

Key Properties & Identities

- Each stage ideally takes one clock cycle in a pipelined processor.
- The slowest stage determines the clock cycle time of the pipeline.

Common Pitfalls

- Misunderstanding the specific function of each pipeline stage.

Standard Problem-Solving Techniques

- Trace an instruction through each stage of the pipeline to understand its execution flow.

Instruction Format

The instruction format defines the layout of bits within a machine instruction. It specifies how the opcode, operands, and addressing mode information are encoded, directly impacting the instruction set's flexibility and the CPU's decoding complexity.

Important Formulas & Concepts

- An instruction typically consists of an **opcode** (operation code) and one or more **operand fields**.
- Operand fields can specify registers, memory addresses, or immediate values.
- **Fixed-length instruction format:** All instructions have the same length. Simplifies fetching and decoding, good for pipelining (RISC).
- **Variable-length instruction format:** Instructions can have different lengths. Allows for more complex instructions and addressing modes, but complicates fetching and decoding (CISC).

Key Properties & Identities

- The number of bits for the opcode determines the maximum number of unique operations.
- The number of bits for operand fields determines the number of registers or the addressable memory range.

Common Pitfalls

- Incorrectly calculating the maximum number of opcodes or addressable memory given a fixed instruction length.

Standard Problem-Solving Techniques

- Given instruction length and operand requirements, calculate the remaining bits for the opcode or vice-versa.

Instruction Set Architecture (ISA)

The ISA is the abstract model of a computer that is visible to a programmer or compiler writer. It defines the set of instructions, registers, memory organization, data types, and the I/O model that the hardware supports.

Important Formulas & Concepts

- Includes the instruction set, register set, memory addressing modes, and interrupt handling mechanisms.
- Forms the interface between software and hardware.

Key Properties & Identities

- ISA defines *what* the processor does, while microarchitecture defines *how* it does it.
- CISC and RISC are two major ISA design philosophies.

Common Pitfalls

- Confusing ISA (the specification) with microarchitecture (the implementation).

Standard Problem-Solving Techniques

- Understand that ISA is the contract that allows software to run on different hardware implementations of the same ISA.

Interrupts

An interrupt is a signal to the CPU indicating an event that requires immediate attention, causing the CPU to temporarily suspend its current task and execute a special routine (Interrupt Service Routine or ISR) to handle the event.

Important Formulas & Concepts

- **Hardware Interrupts:** Generated by I/O devices (e.g., keyboard press, disk completion). Asynchronous.
- **Software Interrupts (Exceptions/Traps):** Generated by program execution (e.g., division by zero, page fault, system call). Synchronous.
- **Interrupt Vector Table:** A table of addresses for ISRs, indexed by interrupt number.

Key Properties & Identities

- Enable efficient I/O handling and error management without constant CPU polling.
- Involve saving CPU state, executing ISR, and restoring CPU state.

Common Pitfalls

- Misunderstanding the sequence of events during interrupt handling (saving context, jumping to ISR, restoring context).

Standard Problem-Solving Techniques

- Trace the flow of control when an interrupt occurs, considering context saving and ISR execution.

Machine Instruction

A machine instruction is a command or operation that a CPU can directly understand and execute. It is represented in binary code and forms the lowest-level programming language, specific to a particular processor's Instruction Set Architecture.

Important Formulas & Concepts

- Composed of an opcode and zero or more operands.
- Each instruction performs a basic operation (e.g., add, load, store, branch).

Key Properties & Identities

- Directly executable by the hardware.
- The fundamental unit of computation for the CPU.

Common Pitfalls

- Confusing machine instructions (binary) with assembly language instructions (mnemonic representation).

Standard Problem-Solving Techniques

- Understand how an assembly instruction translates into its binary machine code equivalent based on the instruction format.

Memory Interfacing

Memory interfacing is the process of connecting memory devices (RAM, ROM) to the CPU. It involves designing the necessary address decoding logic, connecting data and address buses, and generating control signals to ensure proper communication and data transfer.

Important Formulas & Concepts

- **Address Decoding:** Logic gates (decoders, AND gates) used to select the correct memory chip based on the CPU's address lines.
- **Chip Select (CS):** A control signal that enables a specific memory chip.

- **Memory Map:** The allocation of address ranges to different memory devices.

Key Properties & Identities

- Ensures that each memory location has a unique address and that the correct device responds to a CPU request.
- Bus width (data and address) must match between CPU and memory.

Common Pitfalls

- Incorrectly designing address decoding logic, leading to address conflicts or unused address space.

Standard Problem-Solving Techniques

- Given CPU address lines and memory chip sizes, determine the address ranges and design the minimal decoding logic.

Microprogramming

Microprogramming is a technique for implementing the control unit of a CPU by storing a sequence of microinstructions in a special control memory. Each machine instruction is executed by a microprogram, which is a sequence of microinstructions.

Important Formulas & Concepts

- **Microinstruction:** A low-level instruction that controls the individual functional units of the CPU (e.g., enable ALU, load register).
- **Control Store (CS):** Memory where microprograms are stored.

Key Properties & Identities

- Offers flexibility: easier to design complex instruction sets and modify existing ones.
- Slower than hardwired control due to the overhead of fetching and decoding microinstructions.
- Commonly used in CISC processors.

Common Pitfalls

- Confusing microinstructions (internal CPU control) with machine instructions (user-level instructions).

Standard Problem-Solving Techniques

- Understand how a machine instruction is broken down into a sequence of micro-operations controlled by microinstructions.

Pipelining

Pipelining is a technique that allows multiple instructions to be in different stages of execution simultaneously, improving the throughput of a processor. It does not reduce the execution time of a single instruction but increases the rate at which instructions complete.

Important Formulas & Concepts

- **Number of stages:** K
- **Number of instructions:** N
- **Clock cycle time:** T_{cycle} (determined by the slowest stage).
- **Execution time for N instructions (non-pipelined):** $T_{non-pipeline} = N \times K \times T_{cycle}$
- **Execution time for N instructions (ideal pipelined):** $T_{pipeline} = (K + N - 1) \times T_{cycle}$
- **Ideal Speedup (S) for large N:** $S = \frac{N \times K \times T_{cycle}}{(K + N - 1) \times T_{cycle}} \approx K$
- **Throughput (ideal):** $\frac{1}{T_{cycle}}$ instructions per cycle.
- **CPI (Cycles Per Instruction):**

- Ideal $CPI = 1$.
- Actual $CPI = 1 + \text{Stalls per instruction}$

Key Properties & Identities

- Increases instruction throughput, not latency.
- Performance is limited by hazards (structural, data, control).

Common Pitfalls

- Incorrectly calculating execution time when stalls are present.
- Assuming pipelining reduces the execution time of a single instruction.

Standard Problem-Solving Techniques

- Draw pipeline diagrams to visualize instruction flow, identify hazards, and calculate stalls.

Runtime Environment

The runtime environment refers to the state of a program during its execution, encompassing all resources and conditions necessary for the program to run. This includes the CPU's registers, memory (stack, heap, code, data segments), program counter, and operating system services.

Important Formulas & Concepts

- Includes CPU state (registers, PC, flags), memory layout (code, data, heap, stack), and OS resources.

Key Properties & Identities

- Managed by the operating system and supported by the hardware architecture.
- Defines how programs interact with the underlying system.

Common Pitfalls

- Not understanding the distinction between compile-time and run-time aspects of a program.

Standard Problem-Solving Techniques

- Understand how memory is organized and utilized by a program during execution (e.g., stack for function calls, heap for dynamic allocation).

Speedup

Speedup is a measure of how much faster a task executes on an improved system compared to an original system. It quantifies the performance gain achieved by optimizations like pipelining, parallel processing, or using faster components.

Important Formulas & Concepts

- **General Speedup:**

$$\text{Speedup} = \frac{\text{Execution Time}_{\text{old}}}{\text{Execution Time}_{\text{new}}}$$

- **Amdahl's Law:** Calculates the theoretical maximum speedup for a system when only a portion of the task can be improved.

$$S_{\text{overall}} = \frac{1}{(1 - P) + \frac{P}{S_{\text{enhanced}}}}$$

where P is the proportion of the program that can be enhanced, and $S_{enhanced}$ is the speedup of the enhanced part.

- **Pipelining Speedup (for large N):** $S \approx K$ (number of pipeline stages)

Key Properties & Identities

- Amdahl's Law highlights that the sequential portion of a program limits overall speedup.
- Speedup is a dimensionless quantity.

Common Pitfalls

- Incorrectly applying Amdahl's Law, especially regarding the value of P .
- Forgetting to account for overheads or non-ideal conditions in pipelining

Standard Problem-Solving Techniques

- Identify the parallelizable and sequential components of a task when using Amdahl's Law.

Stall

A stall (also known as a bubble or NOP) is a delay introduced into a pipeline to resolve a hazard. When a hazard is detected, the pipeline is temporarily halted for one or more clock cycles, preventing the dependent instruction from proceeding until the hazard is cleared.

Important Formulas & Concepts

- Stalls increase the CPI (Cycles Per Instruction) of the pipeline.
- Number of stalls depends on the type of hazard and the pipeline's ability to forward data.

Key Properties & Identities

- Reduces the effective throughput and speedup of the pipeline.
- A common method to ensure correctness in the presence of data and control hazards.

Common Pitfalls

- Incorrectly calculating the number of stalls required for a given hazard, especially when forwarding is implemented.

Standard Problem-Solving Techniques

- Draw pipeline diagrams and mark the clock cycles where stalls are inserted to resolve dependencies.

Virtual Memory

Virtual memory is a memory management technique that allows a computer to compensate for physical memory shortages by temporarily transferring data from RAM to disk storage. It provides the illusion of a larger, contiguous memory space to programs.

Important Formulas & Concepts

- **Paging:** Divides memory into fixed-size blocks (pages and frames).
- **Page Table:** Maps virtual page numbers to physical frame numbers.
- **TLB (Translation Lookaside Buffer):** A small, fast cache for page table entries.
- **Effective Access Time (EAT) with TLB:**

$$EAT = TLB_{hit_rate} \times (T_{TLB} + T_{memory}) + TLB_{miss_rate} \times (T_{TLB} + 2 \times T_{memory})$$

where T_{TLB} is TLB access time and T_{memory} is main memory access time.

- **Effective Access Time (EAT) with Page Faults:**

$$EAT = (1 - P) \times T_{memory} + P \times T_{page_fault}$$

where P is the page fault rate, T_{memory} is memory access time, and T_{page_fault} is the time to handle a page fault (including disk I/O).

Key Properties & Identities

- Provides memory protection, allows multitasking, and enables programs larger than physical memory.
- Address translation involves converting virtual addresses to physical addresses.

Common Pitfalls

- Incorrectly calculating EAT, especially when considering both TLB misses and page faults.
- Confusing page table entries with TLB entries.

Standard Problem-Solving Techniques

- Trace the address translation process (TLB lookup, page table lookup, page fault handling).

Quick Formula Reference

Topic	Formula/Concept
Average Memory Access Time (AMAT)	- Single-level cache: $AMAT = T_{hit} + MR \times T_{miss_penalty}$ - Two-level cache: $AMAT = T_{L1_hit} + MR_{L1} \times (T_{L2_hit} + MR_{L2} \times T_{main_memory})$
Cache Memory (Address Breakdown)	- Block Offset bits: $\log_2(\text{Block Size})$ - Direct Mapped Index bits: $\log_2(\text{Number of Cache Lines})$ - Set-Associative Index bits: $\log_2(\text{Number of Sets})$ - Tag bits: Total Address Bits – Index Bits – Block Offset Bits - Direct Mapped Index: (Main Memory Block Number) (mod Number of Cache Lines)
Disk Access Time	$Access_Time = Seek_Time + Rotational_Latency + Transfer_Time$ $Average_Rotational_Latency = \frac{1}{2} \times \frac{60}{RPM}$ $Transfer_Time = \frac{\text{Number of Sectors}}{\text{Sectors per Track}} \times \frac{60}{RPM}$
Pipelining Performance	- Execution time (N instructions, K stages, ideal): $T_{pipeline} = (K + N - 1) \times T_{cycle}$ - Ideal Speedup (large N): $S \approx K$ - Actual CPI: $1 + \text{Stalls per instruction}$
Speedup (Amdahl's Law)	$S_{overall} = \frac{1}{(1-P) + \frac{P}{S_{enhanced}}}$
Virtual Memory (EAT)	- With TLB: $EAT = TLB_{hit_rate} \times (T_{TLB} + T_{memory}) + TLB_{miss_rate} \times (T_{TLB} + 2 \times T_{memory})$ - With Page Faults: $EAT = (1 - P) \times T_{memory} + P \times T_{page_fault}$

Important Tips for GATE

- Master Numerical Problems:** A significant portion of COA questions are numerical, especially on Cache Memory (AMAT, address breakdown), Pipelining (execution time, speedup, stalls), Disk Access Time, and Virtual Memory (EAT). Practice these extensively.
- Address Breakdown is Key:** For cache and virtual memory, thoroughly understand how a logical/physical address is divided into Tag, Index/Page Number, and Offset. This is fundamental for many problems.
- Pipeline Diagrams:** Learn to draw and analyze pipeline diagrams for instruction sequences. This is the best way to identify data/control hazards and calculate the number of stalls or benefits of forwarding.

- **Conceptual Clarity:** Don't just memorize definitions. Understand the "why" behind concepts. For example, why RISC is better for pipelining, why cache works (locality), or why DMA is used.
- **Distinguish Similar Concepts:** Be clear on the differences between CISC/RISC, hardwired/microprogrammed control, various addressing modes, and different types of hazards (structural, data, control).
- **Read Questions Carefully:** Pay close attention to units (ns, ms, cycles), specific assumptions (e.g., "no forwarding," "average rotational latency"), and what is being asked (e.g., total time vs. speedup).
- **Amdahl's Law:** This formula frequently appears in questions related to performance improvement. Understand how to apply it correctly, especially identifying the parallelizable portion P .
- **Time Management:** Some numerical problems can be lengthy. If you get stuck or find a calculation too complex, consider moving on and returning later. Look for shortcuts or approximations if applicable.

1.1

Addressing Modes (19)

Practice Test: Test (15Q)

1.1.1 Addressing Modes: GATE CSE 1987 | Question: 1-V



The most relevant addressing mode to write position-independent codes is:

- A. Direct mode B. Indirect mode C. Relative mode D. Indexed mode

gate1987 co-and-architecture addressing-modes easy

Answer key

1.1.2 Addressing Modes: GATE CSE 1988 | Question: 9iii



In the program scheme given below indicate the instructions containing any operand needing relocation for position independent behaviour. Justify your answer.

```

Y = 10
MOV    X(R0), R1
MOV    X, R0
MOV    2(R0), R1
MOV    Y(R0), R5
.
.
.
X :    WORD    0, 0, 0

```

gate1988 normal descriptive co-and-architecture addressing-modes

Answer key

1.1.3 Addressing Modes: GATE CSE 1989 | Question: 2-ii



Match the pairs in the following questions:

(A) Base addressing	(p) Reentrancy
(B) Indexed addressing	(q) Accumulator
(C) Stack addressing	(r) Array
(D) Implied addressing	(s) Position independent

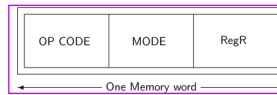
gate1989 match-the-following co-and-architecture addressing-modes easy

Answer key

1.1.4 Addressing Modes: GATE CSE 1993 | Question: 10



The instruction format of a CPU is:



Mode and RegR together specify the operand. RegR specifies a CPU register and Mode specifies an addressing mode. In particular, Mode = 2 specifies that 'the register RegR contains the address of the operand, after fetching the operand, the contents of RegR are incremented by 1'.

An instruction at memory location 2000 specifies Mode = 2 and the RegR refers to program counter (PC).

- A. What is the address of the operand?
- B. Assuming that is a non-jump instruction, what are the contents of PC after the execution of this instruction?

gate1993 co-and-architecture addressing-modes normal descriptive

Answer key

1.1.5 Addressing Modes: GATE CSE 1996 | Question: 1.16, ISRO2016-42



Relative mode of addressing is most relevant to writing:

- A. Co – routines
- B. Position – independent code
- C. Shareable code
- D. Interrupt Handlers

gate1996 co-and-architecture addressing-modes easy isro2016

Answer key

1.1.6 Addressing Modes: GATE CSE 1998 | Question: 1.19



Which of the following addressing modes permits relocation without any change whatsoever in the code?

- A. Indirect addressing
- B. Indexed addressing
- C. Base register addressing
- D. PC relative addressing

gate1998 co-and-architecture addressing-modes easy

Answer key

1.1.7 Addressing Modes: GATE CSE 1999 | Question: 2.23



A certain processor supports only the immediate and the direct addressing modes. Which of the following programming language features cannot be implemented on this processor?

- A. Pointers
- B. Arrays
- C. Records
- D. Recursive procedures with local variable

gate1999 co-and-architecture addressing-modes normal multiple-selects

Answer key

1.1.8 Addressing Modes: GATE CSE 2000 | Question: 1.10



The most appropriate matching for the following pairs

- | | |
|------------------------------|--------------|
| X: Indirect addressing | 1: Loops |
| Y: Immediate addressing | 2: Pointers |
| Z: Auto decrement addressing | 3: Constants |

is

- A. $X - 3, Y - 2, Z - 1$
- B. $X - 1, Y - 3, Z - 2$
- C. $X - 2, Y - 3, Z - 1$
- D. $X - 3, Y - 1, Z - 2$

gatecse-2000 co-and-architecture easy addressing-modes match-the-following

Answer key

1.1.9 Addressing Modes: GATE CSE 2001 | Question: 2.9



Which is the most appropriate match for the items in the first column with the items in the second column:

X. Indirect Addressing	I. Array implementation
Y. Indexed Addressing	II. Writing relocatable code
Z. Base Register Addressing	III. Passing array as parameter

- A. (X, III), (Y, I), (Z, II)
C. (X, III), (Y, II), (Z, I)

- B. (X, II), (Y, III), (Z, I)
D. (X, I), (Y, III), (Z, II)

gatecse-2001 co-and-architecture addressing-modes easy match-the-following

Answer key

1.1.10 Addressing Modes: GATE CSE 2002 | Question: 1.24



In the absolute addressing mode:

- A. the operand is inside the instruction
B. the address of the operand is inside the instruction
C. the register containing the address of the operand is specified inside the instruction
D. the location of the operand is implicit

gatecse-2002 co-and-architecture addressing-modes easy

Answer key

1.1.11 Addressing Modes: GATE CSE 2004 | Question: 20



Which of the following addressing modes are suitable for program relocation at run time?

- I. Absolute addressing
II. Based addressing
III. Relative addressing
IV. Indirect addressing

- A. I and IV
C. II and III

- B. I and II
D. I, II and IV

gatecse-2004 co-and-architecture addressing-modes easy

Answer key

1.1.12 Addressing Modes: GATE CSE 2005 | Question: 65



Consider a three word machine instruction

$ADD A[R_0], @B$

The first operand (destination) " $A[R_0]$ " uses indexed addressing mode with R_0 as the index register. The second operand (source) " $@B$ " uses indirect addressing mode. A and B are memory addresses residing at the second and third words, respectively. The first word of the instruction specifies the opcode, the index register designation and the source and destination addressing modes. During execution of ADD instruction, the two operands are added and stored in the destination (first operand).

The number of memory cycles needed during the execution cycle of the instruction is:

- A. 3 B. 4 C. 5 D. 6

gatecse-2005 co-and-architecture addressing-modes normal

Answer key

1.1.13 Addressing Modes: GATE CSE 2005 | Question: 66



Match each of the high level language statements given on the left hand side with the most natural addressing mode from those listed on the right hand side.

- (1) $A[I] = B[J]$
- (2) `while (*A++);`
- (3) `int temp = *x`

- (a) Indirect addressing
- (b) Indexed addressing
- (c) Auto increment

- A. (1, c), (2, b), (3, a)
- C. (1, b), (2, c), (3, a)

- B. (1, c), (2, c), (3, b)
- D. (1, a), (2, b), (3, c)

gatecse-2005 co-and-architecture addressing-modes match-the-following easy

Answer key

1.1.14 Addressing Modes: GATE CSE 2008 | Question: 33, ISRO2009-80



Which of the following is/are true of the auto-increment addressing mode?

- I. It is useful in creating self-relocating code
- II. If it is included in an Instruction Set Architecture, then an additional ALU is required for effective address calculation
- III. The amount of increment depends on the size of the data item accessed

- A. I only
- B. II only
- C. III only
- D. II and III only

gatecse-2008 addressing-modes co-and-architecture normal isro2009

Answer key

1.1.15 Addressing Modes: GATE CSE 2011 | Question: 21



Consider a hypothetical processor with an instruction of type `LW R1, 20(R2)`, which during execution reads a 32-bit word from memory and stores it in a 32-bit register `R1`. The effective address of the memory location is obtained by the addition of a constant 20 and the contents of register `R2`. Which of the following best reflects the addressing mode implemented by this instruction for the operand in memory?

- A. Immediate addressing
- B. Register addressing
- C. Register Indirect Addressing
- D. Base Indexed Addressing

gatecse-2011 co-and-architecture addressing-modes easy

Answer key

1.1.16 Addressing Modes: GATE CSE 2017 | Set 1 | Question: 11



Consider the C struct defined below:

```
struct data {
    int marks[100];
    char grade;
    int cnumber;
};
struct data student;
```

The base address of student is available in register `R1`. The field `student.grade` can be accessed efficiently using:

- A. Post-increment addressing mode, $(R1) +$
- B. Pre-decrement addressing mode, $-(R1)$
- C. Register direct addressing mode, `R1`
- D. Index addressing mode, $X(R1)$, where X is an offset represented in 2's complement 16-bit representation

gatecse-2017-set1 co-and-architecture addressing-modes

Answer key

1.1.17 Addressing Modes: GATE CSE 2026 | Set 1 | Question: 4



Match each addressing mode in **List I** with a data element or an element of a data structure (in a high-level language) in **List II**:

List I	List II
P. Immediate	1. Element of an array
Q. Indirect	2. Pointer
R. Base with index	3. Element of a record
S. Base with offset/displacement	4. Constant

- A. P – 4, Q – 3, R – 1, S – 2
 B. P – 4, Q – 2, R – 1, S – 3
 C. P – 1, Q – 4, R – 3, S – 2
 D. P – 2, Q – 3, R – 1, S – 4

gatecse-2026-set1 co-and-architecture addressing-modes one-mark

Answer key

1.1.18 Addressing Modes: GATE IT 2006 | Question: 39, ISRO2009-42



Which of the following statements about relative addressing mode is FALSE?

- A. It enables reduced instruction size
 B. It allows indexing of array element with same instruction
 C. It enables easy relocation of data
 D. It enables faster address calculation than absolute addressing

gateit-2006 co-and-architecture addressing-modes normal isro2009

Answer key

1.1.19 Addressing Modes: GATE IT 2006 | Question: 40



The memory locations 1000, 1001 and 1020 have data values 18, 1 and 16 respectively before the following program is executed.

```

MOVI    Rs, 1           ; Move immediate
LOAD    Rd, 1000(Rs)    ; Load from memory
ADDI    Rd, 1000         ; Add immediate
STOREI  0(Rd), 20        ; Store immediate

```

Which of the statements below is TRUE after the program is executed?

- A. Memory location 1000 has value 20
 B. Memory location 1020 has value 20
 C. Memory location 1021 has value 20
 D. Memory location 1001 has value 20

gateit-2006 co-and-architecture addressing-modes normal

Answer key

1.2 Average Memory Access Time (3)

1.2.1 Average Memory Access Time: GATE CSE 1992 | Question: 5-a



The access times of the main memory and the Cache memory, in a computer system, are 500 n sec and 50 nsec, respectively. It is estimated that 80% of the main memory request are for read the rest for write. The hit ratio for the read access only is 0.9 and a write-through policy (where both main and cache memories are updated simultaneously) is used. Determine the average time of the main memory (in ns).

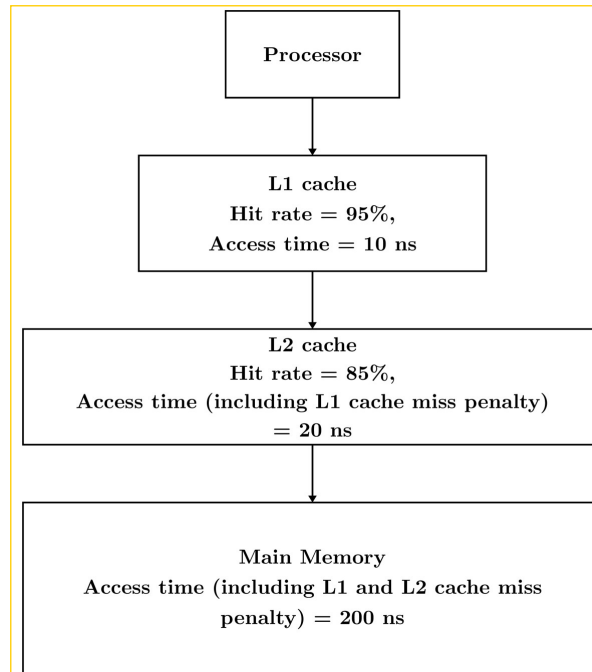
gate1992 co-and-architecture cache-memory normal numerical-answers average-memory-access-time

Answer key

1.2.2 Average Memory Access Time: GATE CSE 2025 | Set 1 | Question: 43



A computer has a memory hierarchy consisting of two-level cache (L1 and L2) and a main memory. If the processor needs to access data from memory, it first looks into L1 cache. If the data is not found in L1 cache, it goes to L2 cache. If it fails to get the data from L2 cache, it goes to main memory, where the data is definitely available. Hit rates and access times of various memory units are shown in the figure. The average memory access time in nanoseconds (ns) is _____. (rounded off to two decimal places)



gategse2025-set1 co-and-architecture cache-memory multilevel-cache average-memory-access-time numerical-answers two-marks

Answer key

1.2.3 Average Memory Access Time: GATE CSE 2025 | Set 2 | Question: 45



Given a computing system with two levels of cache (L1 and L2) and a main memory. The first level (L1) cache access time is 1 nanosecond (ns) and the "hit rate" for L1 cache is 90% while the processor is accessing the data from L1 cache. Whereas, for the second level (L2) cache, the "hit rate" is 80% and the "miss penalty" for transferring data from L2 cache to L1 cache is 10 ns . The "miss penalty" for the data to be transferred from main memory to L2 cache is 100 ns .

Then the average memory access time in this system in nanoseconds is _____. (rounded off to one decimal place)

gategse2025-set2 co-and-architecture cache-memory average-memory-access-time multilevel-cache numerical-answers two-marks

Answer key

1.3 Bit Vector (1)

1.3.1 Bit Vector: GATE CSE 2026 | Set 2 | Question: 43



To keep track of free blocks in a file system, one of the two approaches is generally used - using bitmaps (bit vectors) or using linked lists. Consider that the linked list approach is used to keep track of free blocks in a file system. Assume that the disk size is 16 GB, block size is 2 KB, and block numbers used are 32-bit long. A single pointer of size 4 bytes is used in each block of the list to point to the next block of the list. The number of blocks required to hold the free disk block numbers is _____. (answer in integer)

Note: $1K = 2^{10}$ and $1G = 2^{30}$

gategse-2026-set2 co-and-architecture bit-vector file-system numerical-answers two-marks

1.4

CISC RISC Architecture (2)

1.4.1 CISC RISC Architecture: GATE CSE 1999 | Question: 2.22



The main difference(s) between a CISC and a RISC processor is/are that a RISC processor typically

- A. has fewer instructions
- B. has fewer addressing modes
- C. has more registers
- D. is easier to implement using hard-wired logic

gate1999 co-and-architecture normal cisc-risc-architecture multiple-selects

1.4.2 CISC RISC Architecture: GATE CSE 2018 | Question: 5



Consider the following processor design characteristics:

- I. Register-to-register arithmetic operations only
- II. Fixed-length instruction format
- III. Hardwired control unit

Which of the characteristics above are used in the design of a RISC processor?

- A. I and II only
- B. II and III only
- C. I and III only
- D. I, II and III

gatecse-2018 co-and-architecture cisc-risc-architecture easy one-mark

1.5

Cache Memory (69)

Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (15Q) Test 4 (15Q) Test 5 (15Q) Test 6 (15Q) Test 7 (15Q) Test 8 (4Q)

1.5.1 Cache Memory: GATE CSE 1987 | Question: 4b



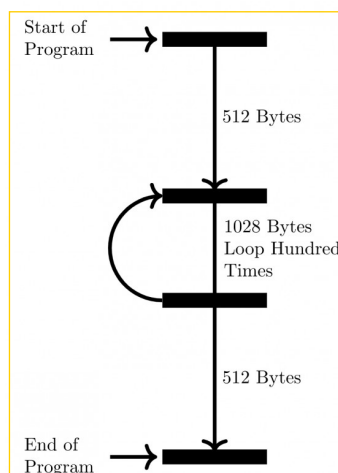
What is cache memory? What is rationale of using cache memory?

gate1987 co-and-architecture cache-memory descriptive

1.5.2 Cache Memory: GATE CSE 1989 | Question: 6a



A certain computer system was designed with cache memory of size 1 Kbytes and main memory size of 256 Kbytes. The cache implementation was fully associative cache with 4 bytes per block. The CPU memory data path was 16 bits and the memory was 2—way interleaved. Each memory read request presents two 16—bit words. A program with the model shown below was run to evaluate the cache design.



Answer the following questions:

- i. What is the hit ratio?
- ii. Suggest a change in the program size of model to improve the hit ratio significantly.

gate1989 descriptive co-and-architecture cache-memory

Answer key

1.5.3 Cache Memory: GATE CSE 1990 | Question: 7a



A block-set associative cache memory consists of 128 blocks divided into four block sets. The main memory consists of 16,384 blocks and each block contains 256 eight bit words.

1. How many bits are required for addressing the main memory?
2. How many bits are needed to represent the TAG, SET and WORD fields?

gate1990 descriptive co-and-architecture cache-memory

Answer key

1.5.4 Cache Memory: GATE CSE 1993 | Question: 11



In the three-level memory hierarchy shown in the following table, p_i denotes the probability that an access request will refer to M_i .

Hierarchy Level (M_i)	Access Time (t_i)	Probability of Access (p_i)	Page Transfer Time (T_i)
M_1	10^{-6}	0.99000	0.001 sec
M_2	10^{-5}	0.00998	0.1 sec
M_3	10^{-4}	0.00002	---

If a miss occurs at level M_i , a page transfer occurs from M_{i+1} to M_i and the average time required for such a page swap is T_i . Calculate the average time t_A required for a processor to read one word from this memory system.

gate1993 co-and-architecture cache-memory normal descriptive

Answer key

1.5.5 Cache Memory: GATE CSE 1995 | Question: 1.6



The principle of locality justifies the use of:

- A. Interrupts B. DMA C. Polling D. Cache Memory

gate1995 co-and-architecture cache-memory easy

Answer key

1.5.6 Cache Memory: GATE CSE 1995 | Question: 2.25



A computer system has a $4K$ word cache organized in block-set-associative manner with 4 blocks per set, 64 words per block. The number of bits in the SET and WORD fields of the main memory address format is:

- A. 15,40 B. 6,4 C. 7,2 D. 4,6

gate1995 co-and-architecture cache-memory normal

Answer key

1.5.7 Cache Memory: GATE CSE 1996 | Question: 26



A computer system has a three-level memory hierarchy, with access time and hit ratios as shown below:

Level 1 (Cache memory)	
Access time $\equiv 50nsec/byte$	
Size	Hit ratio
8M bytes	0.80
16M bytes	0.90
64M bytes	0.95

Level 2 (Main memory)	
Access time $\equiv 200nsec/byte$	
Size	Hit ratio
4M bytes	0.98
16M bytes	0.99
64M bytes	0.995

Level 3	
Access time $\equiv 5\mu sec/byte$	
Size	Hit ratio
260M bytes	1.0

- A. What should be the minimum sizes of level 1 and 2 memories to achieve an average access time of less than $100nsec$?
- B. What is the average access time achieved using the chosen sizes of level 1 and level 2 memories?

gate1996 co-and-architecture cache-memory normal

Answer key

1.5.8 Cache Memory: GATE CSE 1998 | Question: 18



For a set-associative Cache organization, the parameters are as follows:

t_c	Cache Access Time
t_m	Main memory access time
l	Number of sets
b	Block size
$k \times b$	Set size

Calculate the hit ratio for a loop executed 100 times where the size of the loop is $n \times b$, and $n = k \times m$ is a non-zero integer and $1 \leq m \leq l$.

Give the value of the hit ratio for $l = 1$.

gate1998 co-and-architecture cache-memory descriptive

Answer key

1.5.9 Cache Memory: GATE CSE 1999 | Question: 1.22



The main memory of a computer has 2^m blocks while the cache has 2^c blocks. If the cache uses the set associative mapping scheme with 2 blocks per set, then block k of the main memory maps to the set:

- A. $(k \bmod m)$ of the cache
- B. $(k \bmod c)$ of the cache
- C. $(k \bmod 2^c)$ of the cache
- D. $(k \bmod 2^m)$ of the cache

gate1999 co-and-architecture cache-memory normal

Answer key

1.5.10 Cache Memory: GATE CSE 2001 | Question: 1.7, ISRO2008-18



More than one word are put in one cache block to:

- A. exploit the temporal locality of reference in a program
- B. exploit the spatial locality of reference in a program
- C. reduce the miss penalty
- D. none of the above

gatecse-2001 co-and-architecture easy cache-memory isro2008

Answer key

1.5.11 Cache Memory: GATE CSE 2001 | Question: 9



A CPU has $32 - bit$ memory address and a $256 KB$ cache memory. The cache is organized as a $4 - way$ set associative cache with cache block size of 16 bytes.

- A. What is the number of sets in the cache?
- B. What is the size (in bits) of the tag field per cache block?

- C. What is the number and size of comparators required for tag matching?
 D. How many address bits are required to find the byte offset within a cache block?
 E. What is the total amount of extra memory (in bytes) required for the tag bits?

gatecse-2001 co-and-architecture cache-memory normal descriptive

Answer key

1.5.12 Cache Memory: GATE CSE 2002 | Question: 10



In a C program, an array is declared as `float A[2048]`. Each array element is 4 Bytes in size, and the starting address of the array is `0x00000000`. This program is run on a computer that has a direct mapped data cache of size 8 Kbytes, with block (line) size of 16 Bytes.

- A. Which elements of the array conflict with element `A[0]` in the data cache? Justify your answer briefly.
 B. If the program accesses the elements of this array one by one in reverse order i.e., starting with the last element and ending with the first element, how many data cache misses would occur? Justify your answer briefly. Assume that the data cache is initially empty and that no other data or instruction accesses are to be considered.

gatecse-2002 co-and-architecture cache-memory normal descriptive

Answer key

1.5.13 Cache Memory: GATE CSE 2004 | Question: 65



Consider a small two-way set-associative cache memory, consisting of four blocks. For choosing the block to be replaced, use the least recently used (LRU) scheme. The number of cache misses for the following sequence of block addresses is:

8, 12, 0, 12, 8.

- A. 2 B. 3 C. 4 D. 5

gatecse-2004 co-and-architecture cache-memory normal

Answer key

1.5.14 Cache Memory: GATE CSE 2005 | Question: 67



Consider a direct mapped cache of size 32 KB with block size 32 bytes. The CPU generates 32 bit addresses. The number of bits needed for cache indexing and the number of tag bits are respectively,

- A. 10, 17 B. 10, 22 C. 15, 17 D. 5, 17

gatecse-2005 co-and-architecture cache-memory easy

Answer key

1.5.15 Cache Memory: GATE CSE 2006 | Question: 74



Consider two cache organizations. First one is 32 KB 2-way set associative with 32 byte block size, the second is of same size but direct mapped. The size of an address is 32 bits in both cases. A 2-to-1 multiplexer has latency of 0.6 ns while a k -bit comparator has latency of $\frac{k}{10}$ ns. The hit latency of the set associative organization is h_1 while that of direct mapped is h_2 .

The value of h_1 is:

- A. 2.4 ns B. 2.3 ns
 C. 1.8 ns D. 1.7 ns

gatecse-2006 co-and-architecture cache-memory normal

Answer key

1.5.16 Cache Memory: GATE CSE 2006 | Question: 75



Consider two cache organizations. First one is 32 kB 2-way set associative with 32 byte block size, the second is of same size but direct mapped. The size of an address is 32 bits in both cases. A 2-to-1

multiplexer has latency of $0.6ns$ while a k -bit comparator has latency of $\frac{k}{10}ns$. The hit latency of the set associative organization is h_1 while that of direct mapped is h_2 . The value of h_2 is:

- A. $2.4ns$ B. $2.3ns$ C. $1.8ns$ D. $1.7ns$

gatecse-2006 co-and-architecture cache-memory normal

Answer key

1.5.17 Cache Memory: GATE CSE 2006 | Question: 80



A CPU has a $32KB$ direct mapped cache with 128 byte-block size. Suppose A is two dimensional array of size 512×512 with elements that occupy 8 -bytes each. Consider the following two C code segments, $P1$ and $P2$.

$P1$:

```
for (i=0; i<512; i++)
{
    for (j=0; j<512; j++)
    {
        x +=A[i][j];
    }
}
```

$P2$:

```
for (i=0; i<512; i++)
{
    for (j=0; j<512; j++)
    {
        x +=A[j][i];
    }
}
```

$P1$ and $P2$ are executed independently with the same initial state, namely, the array A is not in the cache and i, j, x are in registers. Let the number of cache misses experienced by $P1$ be M_1 and that for $P2$ be M_2 .

The value of M_1 is:

- A. 0 B. 2048 C. 16384 D. 262144

gatecse-2006 co-and-architecture cache-memory normal

Answer key

1.5.18 Cache Memory: GATE CSE 2006 | Question: 81



A CPU has a $32KB$ direct mapped cache with 128 byte-block size. Suppose A is two dimensional array of size 512×512 with elements that occupy 8 -bytes each. Consider the following two C code segments, $P1$ and $P2$.

$P1$:

```
for (i=0; i<512; i++)
{
    for (j=0; j<512; j++)
    {
        x +=A[i][j];
    }
}
```

$P2$:

```
for (i=0; i<512; i++)
{
    for (j=0; j<512; j++)
    {
        x +=A[j][i];
    }
}
```

$P1$ and $P2$ are executed independently with the same initial state, namely, the array A is not in the cache and i, j ,

x are in registers. Let the number of cache misses experienced by P_1 be M_1 and that for P_2 be M_2 .

The value of the ratio $\frac{M_1}{M_2}$:

A. 0

B. $\frac{1}{16}$

C. $\frac{1}{8}$

D. 16

co-and-architecture cache-memory normal gatecse-2006

Answer key

1.5.19 Cache Memory: GATE CSE 2007 | Question: 10



Consider a 4-way set associative cache consisting of 128 lines with a line size of 64 words. The CPU generates a 20-bit address of a word in main memory. The number of bits in the TAG, LINE and WORD fields are respectively:

A. 9, 6, 5

B. 7, 7, 6

C. 7, 5, 8

D. 9, 5, 6

gatecse-2007 co-and-architecture cache-memory normal

Answer key

1.5.20 Cache Memory: GATE CSE 2007 | Question: 80



Consider a machine with a byte addressable main memory of 2^{16} bytes. Assume that a direct mapped data cache consisting of 32 lines of 64 bytes each is used in the system. A 50×50 two-dimensional array of bytes is stored in the main memory starting from memory location $1100H$. Assume that the data cache is initially empty. The complete array is accessed twice. Assume that the contents of the data cache do not change in between the two accesses.

How many data misses will occur in total?

A. 48

B. 50

C. 56

D. 59

gatecse-2007 co-and-architecture cache-memory normal

Answer key

1.5.21 Cache Memory: GATE CSE 2007 | Question: 81



Consider a machine with a byte addressable main memory of 2^{16} bytes. Assume that a direct mapped data cache consisting of 32 lines of 64 bytes each is used in the system. A 50×50 two-dimensional array of bytes is stored in the main memory starting from memory location $1100H$. Assume that the data cache is initially empty. The complete array is accessed twice. Assume that the contents of the data cache do not change in between the two accesses.

Which of the following lines of the data cache will be replaced by new blocks in accessing the array for the second time?

A. line 4 to line 11

B. line 4 to line 12

C. line 0 to line 7

D. line 0 to line 8

gatecse-2007 co-and-architecture cache-memory normal

Answer key

1.5.22 Cache Memory: GATE CSE 2008 | Question: 35



For inclusion to hold between two cache levels L_1 and L_2 in a multi-level cache hierarchy, which of the following are necessary?

- I. L_1 must be write-through cache
- II. L_2 must be a write-through cache
- III. The associativity of L_2 must be greater than that of L_1
- IV. The L_2 cache must be at least as large as the L_1 cache

A. IV only

B. I and IV only

C. I, II and IV only

D. I, II, III and IV

gatecse-2008 co-and-architecture cache-memory normal

Answer key

1.5.23 Cache Memory: GATE CSE 2008 | Question: 71



Consider a machine with a 2-way set associative data cache of size 64Kbytes and block size 16bytes. The cache is managed using 32 bit virtual addresses and the page size is 4Kbytes. A program to be run on this machine begins as follows:

```
double ARR[1024][1024];
int i, j;
/*Initialize array ARR to 0.0 */
for(i = 0; i < 1024; i++)
    for(j = 0; j < 1024; j++)
        ARR[i][j] = 0.0;
```

The size of double is 8Bytes. Array ARR is located in memory starting at the beginning of virtual page 0xFF000 and stored in row major order. The cache is initially empty and no pre-fetching is done. The only data memory references made by the program are those to array ARR.

The total size of the tags in the cache directory is:

- A. 32Kbits B. 34Kbits C. 64Kbits D. 68Kbits

gatecse-2008 co-and-architecture cache-memory normal

Answer key

1.5.24 Cache Memory: GATE CSE 2008 | Question: 72



Consider a machine with a 2-way set associative data cache of size 64 Kbytes and block size 16 bytes. The cache is managed using 32 bit virtual addresses and the page size is 4 Kbytes. A program to be run on this machine begins as follows:

```
double ARR[1024][1024];
int i, j;
/*Initialize array ARR to 0.0 */
for(i = 0; i < 1024; i++)
    for(j = 0; j < 1024; j++)
        ARR[i][j] = 0.0;
```

The size of double is 8 bytes. Array ARR is located in memory starting at the beginning of virtual page 0xFF000 and stored in row major order. The cache is initially empty and no pre-fetching is done. The only data memory references made by the program are those to array ARR.

Which of the following array elements have the same cache index as ARR[0][0]?

- A. ARR[0][4] B. ARR[4][0] C. ARR[0][5] D. ARR[5][0]

gatecse-2008 co-and-architecture cache-memory normal

Answer key

1.5.25 Cache Memory: GATE CSE 2008 | Question: 73



Consider a machine with a 2-way set associative data cache of size 64 Kbytes and block size 16 bytes. The cache is managed using 32 bit virtual addresses and the page size is 4 Kbytes. A program to be run on this machine begins as follows:

```
double ARR[1024][1024];
int i, j;
/*Initialize array ARR to 0.0 */
for(i = 0; i < 1024; i++)
    for(j = 0; j < 1024; j++)
        ARR[i][j] = 0.0;
```

The size of double is 8 bytes. Array ARR is located in memory starting at the beginning of virtual page 0xFF000 and stored in row major order. The cache is initially empty and no pre-fetching is done. The only data memory references made by the program are those to array ARR.

The cache hit ratio for this initialization loop is:

A. 0%

B. 25%

C. 50%

D. 75%

gatecse-2008 co-and-architecture cache-memory normal

Answer key

1.5.26 Cache Memory: GATE CSE 2009 | Question: 29



Consider a 4-way set associative cache (initially empty) with total 16 cache blocks. The main memory consists of 256 blocks and the request for memory blocks are in the following order:

0, 255, 1, 4, 3, 8, 133, 159, 216, 129, 63, 8, 48, 32, 73, 92, 155.

Which one of the following memory block will NOT be in cache if LRU replacement policy is used?

A. 3

B. 8

C. 129

D. 216

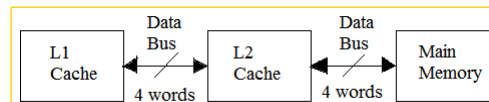
gatecse-2009 co-and-architecture cache-memory normal

Answer key

1.5.27 Cache Memory: GATE CSE 2010 | Question: 48



A computer system has an L1 cache, an L2 cache, and a main memory unit connected as shown below. The block size in L1 cache is 4 words. The block size in L2 cache is 16 words. The memory access times are 2 nanoseconds, 20 nanoseconds and 200 nanoseconds for L1 cache, L2 cache and the main memory unit respectively.



When there is a miss in L1 cache and a hit in L2 cache, a block is transferred from L2 cache to L1 cache. What is the time taken for this transfer?

A. 2 nanoseconds

B. 20 nanoseconds

C. 22 nanoseconds

D. 88 nanoseconds

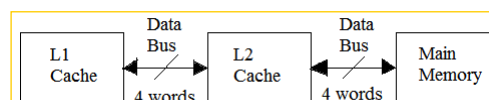
gatecse-2010 co-and-architecture cache-memory normal barc2017

Answer key

1.5.28 Cache Memory: GATE CSE 2010 | Question: 49



A computer system has an L1 cache, an L2 cache, and a main memory unit connected as shown below. The block size in L1 cache is 4 words. The block size in L2 cache is 16 words. The memory access times are 2 nanoseconds, 20 nanoseconds and 200 nanoseconds for L1 cache, L2 cache and the main memory unit respectively.



When there is a miss in both L1 cache and L2 cache, first a block is transferred from main memory to L2 cache, and then a block is transferred from L2 cache to L1 cache. What is the total time taken for these transfers?

A. 222 nanoseconds

B. 888 nanoseconds

C. 902 nanoseconds

D. 968 nanoseconds

gatecse-2010 co-and-architecture cache-memory normal

Answer key

1.5.29 Cache Memory: GATE CSE 2011 | Question: 43



An 8KB direct-mapped write-back cache is organized as multiple blocks, each size of 32-bytes. The processor generates 32-bit addresses. The cache controller contains the tag information for each cache block comprising of the following.

- 1 valid bit
- 1 modified bit
- As many bits as the minimum needed to identify the memory block mapped in the cache.

What is the total size of memory needed at the cache controller to store meta-data (tags) for the cache?

- A. 4864 bits B. 6144 bits C. 6656 bits D. 5376 bits

gatecse-2011 co-and-architecture cache-memory normal

Answer key

1.5.30 Cache Memory: GATE CSE 2012 | Question: 54



A computer has a 256-KByte, 4-way set associative, write back data cache with block size of 32-Bytes. The processor sends 32-bit addresses to the cache controller. Each cache tag directory entry contains, in addition to address tag, 2 valid bits, 1 modified bit and 1 replacement bit. The number of bits in the tag field of an address is

- A. 11 B. 14 C. 16 D. 27

gatecse-2012 co-and-architecture cache-memory normal

Answer key

1.5.31 Cache Memory: GATE CSE 2012 | Question: 55



A computer has a 256-KByte, 4-way set associative, write back data cache with block size of 32 Bytes. The processor sends 32 bit addresses to the cache controller. Each cache tag directory entry contains, in addition to address tag, 2 valid bits, 1 modified bit and 1 replacement bit. The size of the cache tag directory is:

- A. 160 Kbits B. 136 Kbits C. 40 Kbits D. 32 Kbits

normal gatecse-2012 co-and-architecture cache-memory

Answer key

1.5.32 Cache Memory: GATE CSE 2013 | Question: 20



In a k -way set associative cache, the cache is divided into v sets, each of which consists of k lines. The lines of a set are placed in sequence one after another. The lines in set s are sequenced before the lines in set $(s + 1)$. The main memory blocks are numbered 0 onwards. The main memory block numbered j must be mapped to any one of the cache lines from

- A. $(j \bmod v) * k$ to $(j \bmod v) * k + (k - 1)$
 B. $(j \bmod v)$ to $(j \bmod v) + (k - 1)$
 C. $(j \bmod k)$ to $(j \bmod k) + (v - 1)$
 D. $(j \bmod k) * v$ to $(j \bmod k) * v + (v - 1)$

gatecse-2013 co-and-architecture cache-memory normal

Answer key

1.5.33 Cache Memory: GATE CSE 2014 | Set 1 | Question: 44



An access sequence of cache block addresses is of length N and contains n unique block addresses. The number of unique block addresses between two consecutive accesses to the same block address is bounded above by k . What is the miss ratio if the access sequence is passed through a cache of associativity $A \geq k$ exercising least-recently-used replacement policy?

- A. $\left(\frac{n}{N}\right)$ B. $\left(\frac{1}{N}\right)$ C. $\left(\frac{1}{A}\right)$ D. $\left(\frac{k}{n}\right)$

gatecse-2014-set1 co-and-architecture cache-memory normal

Answer key

1.5.34 Cache Memory: GATE CSE 2014 | Set 2 | Question: 43



In designing a computer's cache system, the cache block (or cache line) size is an important parameter. Which one of the following statements is correct in this context?

- A. A smaller block size implies better spatial locality
- B. A smaller block size implies a smaller cache tag and hence lower cache tag overhead
- C. A smaller block size implies a larger cache tag and hence lower cache hit time
- D. A smaller block size incurs a lower cache miss penalty

gatecse-2014-set2 co-and-architecture cache-memory normal

Answer key

1.5.35 Cache Memory: GATE CSE 2014 | Set 2 | Question: 44



If the associativity of a processor cache is doubled while keeping the capacity and block size unchanged, which one of the following is guaranteed to be NOT affected?

- A. Width of tag comparator
- B. Width of set index decoder
- C. Width of way selection multiplexer
- D. Width of processor to main memory data bus

gatecse-2014-set2 co-and-architecture cache-memory normal

Answer key

1.5.36 Cache Memory: GATE CSE 2014 | Set 2 | Question: 9



A 4-way set-associative cache memory unit with a capacity of 16 KB is built using a block size of 8 words. The word length is 32 bits. The size of the physical address space is 4 GB. The number of bits for the TAG field is ____.

gatecse-2014-set2 co-and-architecture cache-memory numerical-answers normal

Answer key

1.5.37 Cache Memory: GATE CSE 2014 | Set 3 | Question: 44



The memory access time is 1 nanosecond for a read operation with a hit in cache, 5 nanoseconds for a read operation with a miss in cache, 2 nanoseconds for a write operation with a hit in cache and 10 nanoseconds for a write operation with a miss in cache. Execution of a sequence of instructions involves 100 instruction fetch operations, 60 memory operand read operations and 40 memory operand write operations. The cache hit-ratio is 0.9. The average memory access time (in nanoseconds) in executing the sequence of instructions is ____.

gatecse-2014-set3 co-and-architecture cache-memory numerical-answers normal

Answer key

1.5.38 Cache Memory: GATE CSE 2015 | Set 2 | Question: 24



Assume that for a certain processor, a read request takes 50 nanoseconds on a cache miss and 5 nanoseconds on a cache hit. Suppose while running a program, it was observed that 80% of the processor's read requests result in a cache hit. The average read access time in nanoseconds is ____.

gatecse-2015-set2 co-and-architecture cache-memory easy numerical-answers

Answer key

1.5.39 Cache Memory: GATE CSE 2015 | Set 3 | Question: 14



Consider a machine with a byte addressable main memory of 2^{20} bytes, block size of 16 bytes and a direct mapped cache having 2^{12} cache lines. Let the addresses of two consecutive bytes in main memory be $(E201F)_{16}$ and $(E2020)_{16}$. What are the tag and cache line addresses (in hex) for main memory address $(E201F)_{16}$?

- A. E, 201
- B. F, 201
- C. E, E20
- D. 2, 01F

gatecse-2015-set3 co-and-architecture cache-memory normal

Answer key

1.5.40 Cache Memory: GATE CSE 2016 | Set 2 | Question: 32



The width of the physical address on a machine is 40 bits. The width of the tag field in a 512 KB 8-way set associative cache is _____ bits.

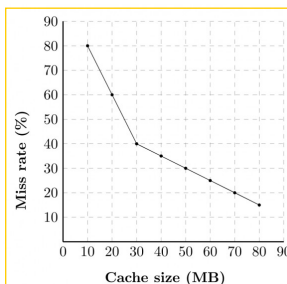
gatecse-2016-set2 co-and-architecture cache-memory normal numerical-answers

Answer key

1.5.41 Cache Memory: GATE CSE 2016 | Set 2 | Question: 50



A file system uses an in-memory cache to cache disk blocks. The miss rate of the cache is shown in the figure. The latency to read a block from the cache is 1 ms and to read a block from the disk is 10 ms. Assume that the cost of checking whether a block exists in the cache is negligible. Available cache sizes are in multiples of 10 MB.



The smallest cache size required to ensure an average read latency of less than 6 ms is _____ MB.

gatecse-2016-set2 co-and-architecture cache-memory normal numerical-answers

Answer key

1.5.42 Cache Memory: GATE CSE 2017 | Set 1 | Question: 25



Consider a two-level cache hierarchy with L_1 and L_2 caches. An application incurs 1.4 memory accesses per instruction on average. For this application, the miss rate of L_1 cache is 0.1; the L_2 cache experiences, on average, 7 misses per 1000 instructions. The miss rate of L_2 expressed correct to two decimal places is _____.

gatecse-2017-set1 co-and-architecture cache-memory numerical-answers

Answer key

1.5.43 Cache Memory: GATE CSE 2017 | Set 1 | Question: 54



A cache memory unit with capacity of N words and block size of B words is to be designed. If it is designed as a direct mapped cache, the length of the TAG field is 10 bits. If the cache unit is now designed as a 16-way set-associative cache, the length of the TAG field is _____ bits.

gatecse-2017-set1 co-and-architecture cache-memory normal numerical-answers

Answer key

1.5.44 Cache Memory: GATE CSE 2017 | Set 2 | Question: 29



In a two-level cache system, the access times of L_1 and L_2 caches are 1 and 8 clock cycles, respectively. The miss penalty from the L_2 cache to main memory is 18 clock cycles. The miss rate of L_1 cache is twice that of L_2 . The average memory access time (AMAT) of this cache system is 2 cycles. The miss rates of L_1 and L_2 respectively are

- A. 0.111 and 0.056 B. 0.056 and 0.111 C. 0.0892 and 0.1784 D. 0.1784 and 0.0892

gatecse-2017-set2 cache-memory co-and-architecture normal

Answer key

1.5.45 Cache Memory: GATE CSE 2017 | Set 2 | Question: 45



The read access times and the hit ratios for different caches in a memory hierarchy are as given below:

Cache	Read access time (in nanoseconds)	Hit ratio
I-cache	2	0.8
D-cache	2	0.9
L2-cache	8	0.9

The read access time of main memory is 90 nanoseconds. Assume that the caches use the referred-word-first read policy and the write-back policy. Assume that all the caches are direct mapped caches. Assume that the dirty bit is always 0 for all the blocks in the caches. In execution of a program, 60% of memory reads are for instruction fetch and 40% are for memory operand fetch. The average read access time in nanoseconds (up to 2 decimal places) is _____

gatecse-2017-set2 co-and-architecture cache-memory numerical-answers

Answer key

1.5.46 Cache Memory: GATE CSE 2017 | Set 2 | Question: 53



Consider a machine with a byte addressable main memory of 2^{32} bytes divided into blocks of size 32 bytes. Assume that a direct mapped cache having 512 cache lines is used with this machine. The size of the tag field in bits is _____

gatecse-2017-set2 co-and-architecture cache-memory numerical-answers

Answer key

1.5.47 Cache Memory: GATE CSE 2018 | Question: 34



The size of the physical address space of a processor is 2^P bytes. The word length is 2^W bytes. The capacity of cache memory is 2^N bytes. The size of each cache block is 2^M words. For a K -way set-associative cache memory, the length (in number of bits) of the tag field is

- A. $P - N - \log_2 K$
 B. $P - N + \log_2 K$
 C. $P - N - M - W - \log_2 K$
 D. $P - N - M - W + \log_2 K$

gatecse-2018 co-and-architecture cache-memory normal two-marks

Answer key

1.5.48 Cache Memory: GATE CSE 2019 | Question: 1



A certain processor uses a fully associative cache of size 16 kB. The cache block size is 16 bytes. Assume that the main memory is byte addressable and uses a 32-bit address. How many bits are required for the Tag and the Index fields respectively in the addresses generated by the processor?

- A. 24 bits and 0 bits
 B. 28 bits and 4 bits
 C. 24 bits and 4 bits
 D. 28 bits and 0 bits

gatecse-2019 co-and-architecture cache-memory normal one-mark

Answer key

1.5.49 Cache Memory: GATE CSE 2019 | Question: 45



A certain processor deploys a single-level cache. The cache block size is 8 words and the word size is 4 bytes. The memory system uses a 60-MHz clock. To service a cache miss, the memory controller first takes 1 cycle to accept the starting address of the block, it then takes 3 cycles to fetch all the eight words of the block, and finally transmits the words of the requested block at the rate of 1 word per cycle. The maximum bandwidth for the memory system when the program running on the processor issues a series of read operations is _____ $\times 10^6$ bytes/sec.

gatecse-2019 numerical-answers co-and-architecture cache-memory two-marks

Answer key

1.5.50 Cache Memory: GATE CSE 2020 | Question: 21



A direct mapped cache memory of 1 MB has a block size of 256 bytes. The cache has an access time of 3 ns and a hit rate of 94%. During a cache miss, it takes 20 ns to bring the first word of a block from the main memory, while each subsequent word takes 5 ns. The word size is 64 bits. The average memory access time in ns (round off to 1 decimal place) is _____.

gatecse-2020 numerical-answers co-and-architecture cache-memory one-mark

Answer key

1.5.51 Cache Memory: GATE CSE 2020 | Question: 30



A computer system with a word length of 32 bits has a 16 MB byte-addressable main memory and a 64 KB 4-way set associative cache memory with a block size of 256 bytes. Consider the following four physical addresses represented in hexadecimal notation.

- A1 = 0x42C8A4
- A2 = 0x546888
- A3 = 0x6A289C
- A4 = 0x5E4880

Which one of the following is TRUE?

- A. A1 and A4 are mapped to different cache sets.
- B. A2 and A3 are mapped to the same cache set.
- C. A3 and A4 are mapped to the same cache set.
- D. A1 and A3 are mapped to the same cache set.

gatecse-2020 co-and-architecture cache-memory two-marks

Answer key

1.5.52 Cache Memory: GATE CSE 2021 | Set 1 | Question: 22



Consider a computer system with a byte-addressable primary memory of size 2^{32} bytes. Assume the computer system has a direct-mapped cache of size 32 KB ($1 \text{ KB} = 2^{10}$ bytes), and each cache block is of size 64 bytes.

The size of the tag field is _____ bits.

gatecse-2021-set1 co-and-architecture cache-memory numerical-answers one-mark

Answer key

1.5.53 Cache Memory: GATE CSE 2021 | Set 2 | Question: 19



Consider a set-associative cache of size 2KB ($1 \text{ KB} = 2^{10}$ bytes) with cache block size of 64 bytes. Assume that the cache is byte-addressable and a 32-bit address is used for accessing the cache. If the width of the tag field is 22 bits, the associativity of the cache is _____.

gatecse-2021-set2 numerical-answers co-and-architecture cache-memory one-mark

Answer key

1.5.54 Cache Memory: GATE CSE 2021 | Set 2 | Question: 27



Assume a two-level inclusive cache hierarchy, L_1 and L_2 , where L_2 is the larger of the two. Consider the following statements.

- S_1 : Read misses in a write through L_1 cache do not result in writebacks of dirty lines to the L_2
- S_2 : Write allocate policy *must* be used in conjunction with write through caches and no-write allocate policy is used with writeback caches.

Which of the following statements is correct?

- A. S_1 is true and S_2 is false
C. S_1 is true and S_2 is true

- B. S_1 is false and S_2 is true
D. S_1 is false and S_2 is false

gatecse-2021-set2 co-and-architecture cache-memory two-marks

Answer key

1.5.55 Cache Memory: GATE CSE 2022 | Question: 14



Let WB and WT be two set associative cache organizations that use LRU algorithm for cache block replacement. WB is a write back cache and WT is a write through cache. Which of the following statements is/are FALSE?

- A. Each cache block in WB and WT has a dirty bit.
B. Every write hit in WB leads to a data transfer from cache to main memory.
C. Eviction of a block from WT will not lead to data transfer from cache to main memory.
D. A read miss in WB will never lead to eviction of a dirty block from WB.

gatecse-2022 co-and-architecture cache-memory multiple-selects one-mark

Answer key

1.5.56 Cache Memory: GATE CSE 2022 | Question: 23



A cache memory that has a hit rate of 0.8 has an access latency 10 ns and miss penalty 100 ns. An optimization is done on the cache to reduce the miss rate. However, the optimization results in an increase of cache access latency to 15 ns, whereas the miss penalty is not affected. The minimum hit rate (rounded off to two decimal places) needed after the optimization such that it should not increase the average memory access time is _____.

gatecse-2022 numerical-answers co-and-architecture cache-memory one-mark

Answer key

1.5.57 Cache Memory: GATE CSE 2023 | Question: 54



An 8-way set associative cache of size 64 KB ($1 \text{ KB} = 1024 \text{ bytes}$) is used in a system with 32-bit address. The address is sub-divided into TAG, INDEX, and BLOCK OFFSET.

The number of bits in the TAG is _____.

gatecse-2023 co-and-architecture cache-memory numerical-answers two-marks

Answer key

1.5.58 Cache Memory: GATE CSE 2024 | Set 1 | Question: 43



Consider two set-associative cache memory architectures: WBC, which uses the write back policy, and WTC, which uses the write through policy. Both of them use the LRU (Least Recently Used) block replacement policy. The cache memory is connected to the main memory. Which of the following statements is/are TRUE?

- A. A read miss in WBC never evicts a dirty block
B. A read miss in WTC never triggers a write back operation of a cache block to main memory
C. A write hit in WBC can modify the value of the dirty bit of a cache block
D. A write miss in WTC always writes the victim cache block to main memory before loading the missed block to the cache

gatecse-2024-set1 co-and-architecture cache-memory multiple-selects two-marks

Answer key

1.5.59 Cache Memory: GATE CSE 2024 | Set 1 | Question: 46



A given program has 25% load/store instructions. Suppose the ideal CPI (cycles per instruction) without any memory stalls is 2. The program exhibits 2% miss rate on instruction cache and 8% miss rate on data cache. The miss penalty is 100 cycles. The speedup (rounded off to two decimal places) achieved with a perfect cache (i.e., with NO data or instruction cache misses) is _____.

gatecse-2024-set1 numerical-answers co-and-architecture cache-memory two-marks

Answer key

1.5.60 Cache Memory: GATE CSE 2026 | Set 1 | Question: 28



The size of the physical address space of a processor is 2^{32} bytes. The capacity of a cache memory unit is 2^{23} bytes. The cache block size is 128 bytes. The cache memory unit can be built as a direct mapped cache or as a K -way set-associative cache, where $K = 2^L$ and $L \in \{1, 2, 3\}$. Let the length of the TAG field be M bits for the direct mapped cache, and N bits for the set-associative cache.

Which one of the following options is true?

- A. $N = M + L$ B. $N = M - L$ C. $N = M + K$ D. $N = M - K$

gatecse-2026-set1 co-and-architecture two-marks cache-memory

Answer key

1.5.61 Cache Memory: GATE CSE 2026 | Set 1 | Question: 44



Consider a system that has a cache memory unit and a memory management unit (MMU). The address input to the cache memory is a physical address. The MMU has a translation lookaside buffer (TLB). Assume that when a page is evicted from the main memory, the corresponding blocks in the cache are marked as invalid.

For a given memory reference, which of the following sequences of events can NEVER happen?

- A. TLB miss, Page table hit, Cache hit
B. TLB hit, Page table miss, Cache hit
C. TLB miss, Page table miss, Cache hit
D. TLB miss, Page table miss, Cache miss

gatecse-2026-set1 co-and-architecture cache-memory two-marks multiple-selects

Answer key

1.5.62 Cache Memory: GATE IT 2004 | Question: 12, ISRO2016-77



Consider a system with 2 level cache. Access times of Level 1 cache, Level 2 cache and main memory are 1 ns, 10 ns, and 500 ns respectively. The hit rates of Level 1 and Level 2 caches are 0.8 and 0.9 respectively. What is the average access time of the system ignoring the search time within the cache?

- A. 13.0 B. 12.8 C. 12.6 D. 12.4

gateit-2004 co-and-architecture cache-memory normal isro2016

Answer key

1.5.63 Cache Memory: GATE IT 2004 | Question: 48



Consider a fully associative cache with 8 cache blocks (numbered 0 – 7) and the following sequence of memory block requests:

4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

If LRU replacement policy is used, which cache block will have memory block 7?

- A. 4 B. 5 C. 6 D. 7

gateit-2004 co-and-architecture cache-memory normal

Answer key

1.5.64 Cache Memory: GATE IT 2005 | Question: 61



Consider a 2-way set associative cache memory with 4 sets and total 8 cache blocks (0 – 7) and a main memory with 128 blocks (0 – 127). What memory blocks will be present in the cache after the following sequence of memory block references if LRU policy is used for cache block replacement. Assuming that initially the cache did not have any memory block from the current job?

0 5 3 9 7 0 16 55

A. 0 3 5 7 16 55

C. 0 5 7 9 16 55

B. 0 3 5 7 9 16 55

D. 3 5 7 9 16 55

gateit-2005 co-and-architecture cache-memory normal

Answer key

1.5.65 Cache Memory: GATE IT 2006 | Question: 42



A cache line is 64 bytes. The main memory has latency 32 ns and bandwidth 1 GBytes/s. The time required to fetch the entire cache line from the main memory is:

A. 32 ns

B. 64 ns

C. 96 ns

D. 128 ns

gateit-2006 co-and-architecture cache-memory normal

Answer key

1.5.66 Cache Memory: GATE IT 2006 | Question: 43



A computer system has a level-1 instruction cache (I-cache), a level-1 data cache (D-cache) and a level-2 cache (L2-cache) with the following specifications:

	Capacity	Mapping Method	Block Size
I-Cache	4K words	Direct mapping	4 words
D-Cache	4K words	2-way set associative mapping	4 words
L2-Cache	64K words	4-way set associative mapping	16 words

The length of the physical address of a word in the main memory is 30 bits. The capacity of the tag memory in the I-cache, D-cache and L2-cache is, respectively,

A. 1 K x 18-bit, 1 K x 19-bit, 4 K x 16-bit

B. 1 K x 16-bit, 1 K x 19-bit, 4 K x 18-bit

C. 1 K x 16-bit, 512 x 18-bit, 1 K x 16-bit

D. 1 K x 18-bit, 512 x 18-bit, 1 K x 18-bit

gateit-2006 co-and-architecture cache-memory normal

Answer key

1.5.67 Cache Memory: GATE IT 2007 | Question: 37



Consider a Direct Mapped Cache with 8 cache blocks (numbered 0 – 7). If the memory block requests are in the following order

3, 5, 2, 8, 0, 63, 9, 16, 20, 17, 25, 18, 30, 24, 2, 63, 5, 82, 17, 24.

Which of the following memory blocks will not be in the cache at the end of the sequence?

A. 3

B. 18

C. 20

D. 30

gateit-2007 co-and-architecture cache-memory normal

Answer key

1.5.68 Cache Memory: GATE IT 2008 | Question: 80



Consider a computer with a 4-ways set-associative mapped cache of the following characteristics: a total of 1 MB of main memory, a word size of 1 byte, a block size of 128 words and a cache size of 8 KB.

The number of bits in the TAG, SET and WORD fields, respectively are:

A. 7,6,7

B. 8,5,7

C. 8,6,6

D. 9,4,7

gateit-2008 co-and-architecture cache-memory normal

Answer key

1.5.69 Cache Memory: GATE IT 2008 | Question: 81



Consider a computer with a 4-ways set-associative mapped cache of the following characteristics: a total of 1 MB of main memory, a word size of 1 byte, a block size of 128 words and a cache size of 8 KB.

While accessing the memory location 0C795H by the CPU, the contents of the TAG field of the corresponding cache line is:

A. 000011000

B. 110001111

C. 00011000

D. 110010101

gateit-2008 co-and-architecture cache-memory normal

Answer key

1.6 Conflict Misses (1)

1.6.1 Conflict Misses: GATE CSE 2017 | Set 1 | Question: 51



Consider a 2-way set associative cache with 256 blocks and uses LRU replacement. Initially the cache is empty. Conflict misses are those misses which occur due to the contention of multiple blocks for the same cache set. Compulsory misses occur due to first time access to the block. The following sequence of access to memory blocks:

{0, 128, 256, 128, 0, 128, 256, 128, 1, 129, 257, 129, 1, 129, 257, 129}

is repeated 10 times. The number of conflict misses experienced by the cache is _____.

gatecse-2017-set1 co-and-architecture cache-memory conflict-misses normal numerical-answers

Answer key

1.7 Control Unit (1)

1.7.1 Control Unit: GATE CSE 1987 | Question: 1-vi



A microprogrammed control unit

- A. Is faster than a hard-wired control unit.
- B. Facilitates easy implementation of new instruction.
- C. Is useful when very small programs are to be run.
- D. Usually refers to the control unit of a microprocessor.

gate1987 co-and-architecture control-unit microprogramming

Answer key

1.8 DMA (8)

Practice Test: Test (12Q)

1.8.1 DMA: GATE CSE 2004 | Question: 68



A hard disk with a transfer rate of 10 Mbytes/second is constantly transferring data to memory using DMA. The processor runs at 600 MHz, and takes 300 and 900 clock cycles to initiate and complete DMA transfer respectively. If the size of the transfer is 20 Kbytes, what is the percentage of processor time consumed for the transfer operation?

A. 5.0%

B. 1.0%

C. 0.5%

D. 0.1%

gatecse-2004 dma normal co-and-architecture

Answer key